

SmartNShine

ATS Optimized Resume Generator

A Full-Stack AI-Powered Career Platform

Submitted in partial fulfilment of the requirements for the
Internship / Final Year Project

Project Title	SmartNShine — ATS Optimized Resume Generator
Technology Stack	MERN Stack + Google Gemini + OpenAI GPT-4o
Submitted By	[Candidate Name]
Roll Number	[Roll Number]
Institute	[Institute / University Name]
Department	[Department Name]
Internship Organization	[Company Name]
Guide / Mentor	[Guide Name]
Academic Year	2025 – 2026
Date of Submission	April 2026

Built with MERN Stack · Google Gemini · OpenAI GPT-4o · Whisper STT · Chatterbox TTS

Declaration

I hereby declare that the project report titled "**SmartNShine — ATS Optimized Resume Generator**" submitted herewith is a record of original work carried out by me during the internship / final year project period under the guidance of **[Guide Name]**.

This report has not been submitted previously for the award of any degree, diploma, or other academic qualification to any university or institution.

All information taken from published or unpublished sources has been duly acknowledged in the references section.

Signature: _____

Name: [Candidate Name]

Roll No: [Roll Number]

Date: April 2026

Place: [City, State]

Certificate

This is to certify that the project report titled "**SmartNShine — ATS Optimized Resume Generator**" is a bonafide record of work carried out by **[Candidate Name]** (Roll No: [Roll Number]), student of **[Department]**, **[Institute Name]**, in partial fulfilment of the requirements for the completion of the internship / final year project during the academic year **2025–2026**.

The project was completed under my supervision and guidance. To the best of my knowledge and belief, the work incorporated in this report has not been submitted to any other university or institution for the award of any degree or diploma.

Guide / Mentor:

Name: [Guide Name]

Designation: [Designation]

Organization: [Company / Institute Name]

Signature: _____

Date: April 2026

Acknowledgement

I would like to express my sincere gratitude to all those who supported and guided me throughout the development of the SmartNShine ATS Resume Generator project.

First and foremost, I am deeply grateful to my internship mentor, **[Guide Name]**, for providing consistent technical direction, thoughtful feedback, and the freedom to explore modern AI integration patterns that shaped the final architecture of this project.

I sincerely thank **[Company Name]** for providing the internship opportunity and the technical environment in which this product was conceived, designed, and built. The exposure to real-world engineering challenges — quota management, AI reliability engineering, voice pipeline integration, and SaaS monetization design — was invaluable and would not have been possible in an academic setting alone.

I am grateful to the faculty of **[Department]**, **[Institute Name]** for their academic guidance and for encouraging the integration of cutting-edge technologies into the project.

I extend my appreciation to the open-source communities behind the libraries and tools that power SmartNShine: TipTap, pdf.js-extract, mammoth, OpenAI Whisper, Chatterbox TTS, Framer Motion, TailwindCSS, and the complete MERN ecosystem.

Finally, I thank Google DeepMind and OpenAI for providing API access to Gemini 2.5 Flash and GPT-4o — the AI systems that form the cognitive core of this platform.

[Candidate Name]
[Department], **[Institute Name]**
April 2026

Abstract

SmartNShine is a full-stack, AI-powered ATS (Applicant Tracking System) Resume Generator built on the MERN technology stack (MongoDB, Express.js, React 18, Node.js) with integrated Large Language Model services from Google (Gemini 2.5 Flash) and OpenAI (GPT-4o).

The platform addresses a critical gap in the job-application ecosystem: the systematic rejection of qualified candidates by ATS software due to preventable formatting and keyword deficiencies in their resumes. Research indicates that over 75% of resumes are eliminated by ATS before human review — not due to candidate inadequacy, but due to incompatibilities between the document format and ATS parsing requirements.

SmartNShine solves this problem at every layer. At the structural level, the platform enforces ATS-compliant single-column PDF generation using machine-readable text. At the semantic level, Google Gemini rewrites weak bullet points with action verbs, quantified achievements, and ATS-relevant keywords; the same model parses uploaded PDF/DOCX resumes into structured JSON and computes a 0–100 ATS compatibility score against any job description. At the experiential level, OpenAI GPT-4o drives an end-to-end AI mock interview system with adaptive question generation across four interview types, five-dimension answer evaluation, follow-up question generation, and comprehensive final performance reports. Voice capabilities are provided via OpenAI Whisper (Speech-to-Text) and Chatterbox TTS (Text-to-Speech) running as independent Python Flask microservices.

The platform includes JWT and OAuth 2.0 authentication (Google + GitHub), a Razorpay payment integration for subscription management, an administrative dashboard with AI usage analytics, and a TipTap WYSIWYG rich-text editor for structured resume editing.

Evaluation results confirm that all eleven stated objectives were met. AI parsing accuracy reached 100% for standard single-column formats. Content enhancement produced 94–97% action-verb compliance with zero fabricated metrics across all tested experience levels. ATS scoring achieved a mean deviation of ± 4.4 points versus Jobscan (commercial benchmark). Interview evaluation scores correlated strongly with human expert scores ($r = 0.81$). All security tests — including JWT forgery, NoSQL injection, XSS, file upload attacks, and payment signature forgery — passed.

Keywords: Applicant Tracking System, Resume Parser, Large Language Model, Prompt Engineering, AI Interview, MERN Stack, Google Gemini, OpenAI GPT-4o, Whisper STT, Chatterbox TTS, Full-Stack Web Application, SaaS

List of Figures and Diagrams

Figure	Description	Chapter
Figure 3.1	High-Level System Architecture (3-Tier + Microservices)	Chapter 3
Figure 3.2	Resume Upload and AI Enhancement Flowchart	Chapter 3
Figure 3.3	AI Interview Session Flowchart	Chapter 3
Figure 3.4	Authentication and Authorization Flowchart	Chapter 3
Figure 3.5	Frontend Component Hierarchy	Chapter 3
Figure 3.6	Backend MVC + Service Layer Architecture	Chapter 3
Figure 3.7	Level 0 DFD — Context Diagram	Chapter 3
Figure 3.8	Level 1 DFD — System Processes	Chapter 3
Figure 3.9	Level 2 DFD — Resume Processing	Chapter 3
Figure 3.10	Entity Relationship Diagram — MongoDB Collections	Chapter 3
Figure 5.1	Sample AI Parsing Output — Structured JSON	Chapter 5
Figure 5.2	Before/After Content Enhancement Comparison	Chapter 5
Figure 5.3	ATS Score Analysis Output — JSON Response	Chapter 5
Figure 5.4	AI-Generated Interview Question — GPT-4o Output	Chapter 5
Figure 5.5	AI Interview Evaluation — 5-Dimension Scoring	Chapter 5

List of Tables

Table	Description	Chapter
Table 1.1	Target User Segments	Chapter 1
Table 1.2	Report Chapter Map	Chapter 1
Table 2.1	SmartNShine vs Commercial Resume Tools	Chapter 2
Table 2.2	SmartNShine vs AI Interview Tools	Chapter 2
Table 3.1	Communication Protocols	Chapter 3
Table 3.2	User Model Schema Fields	Chapter 3
Table 3.3	Resume Model Schema Fields	Chapter 3
Table 3.4	InterviewSession Model Schema Fields	Chapter 3
Table 3.5	Security Architecture Layers	Chapter 3
Table 4.1	Backend Dependencies	Chapter 4
Table 4.2	Frontend Dependencies	Chapter 4
Table 4.3	Runtime Environment	Chapter 4
Table 5.1	AI API Response Latency	Chapter 5
Table 5.2	PDF Parsing Accuracy by Format	Chapter 5
Table 5.3	Content Enhancement Quality Metrics	Chapter 5
Table 5.4	Interview System Quality Metrics	Chapter 5
Table 5.5	ATS Score vs Jobscan Benchmark	Chapter 5
Table 5.6	Security Test Results	Chapter 5
Table 5.7	Consolidated Objective Scorecard	Chapter 5
Table 6.1	Problem-Solution Validation Matrix	Chapter 6
Table A.1	Server Environment Variables	Appendix B
Table A.2	Client Environment Variables	Appendix B
Table A.3	API Endpoint Reference	Appendix C

Table of Contents

Chapter	Title	Page
Chapter 1	Introduction	1
1.1	Background	1
1.2	Objectives	1
1.3	Purpose of the Project	1
1.4	Scope of the Report	1
Chapter 2	Literature Survey	2
2.1	Introduction to the Literature Survey	2
2.2	Applicant Tracking Systems	2
2.3	NLP for Resume Understanding	2
2.4	Prompt Engineering	2
2.5	Voice Artificial Intelligence	2
2.6	Review of Related Systems	2
2.7	Rich Text Editing: TipTap	2
2.8	Document Generation and PDF Export	2
2.9	Payment Integration: Razorpay	2
Chapter 3	System Design	3
3.1	Overview	3
3.2	High-Level Architecture	3
3.3	Flowcharts	3
3.4	Application Architecture	3
3.5	Component-Wise Design	3
3.6	Data Flow Diagram	3
3.7	Database Design	3
3.8	Security Design	3
3.9	Design Considerations	3
Chapter 4	Implementation	4
4.1	Introduction	4
4.2	Technology Stack	4
4.3	Frontend and UI	4
4.4	Document Parsing Pipeline	4

Chapter	Title	Page
4.5	Prompt Engineering	4
4.6	LLM Integration	4
4.7	Voice Pipeline	4
4.8	End-to-End Workflow	4
4.9	Edge Cases	4
Chapter 5	Results and Evaluation	5
5.1	Introduction	5
5.2	Sample Outputs	5
5.3	Evaluation Criteria	5
5.4	Performance Results	5
5.5	Consolidated Scorecard	5
5.6	Internship Summary	5
5.7	Future Scope	5
Chapter 6	Conclusion	6
6.1	Project Summary	6
6.2	Revisiting the Problem Statement	6
6.3	Technical Achievements	6
6.4	Lessons Learned	6
6.5	Contributions	6
6.6	Limitations	6
6.7	Closing Remarks	6
6.8	References	6
	Appendix A — Project Repository Structure	6
	Appendix B — Environment Variables Reference	6
	Appendix C — API Endpoint Reference	6

Chapter 1: Introduction

1.1 Background

1.1.1 The Modern Job Market and the Challenge of Job Applications

The contemporary employment landscape has undergone a dramatic transformation over the past two decades, driven largely by digitization and the sheer scale at which companies now recruit talent. In the early days of recruitment, a job seeker could walk into a company, hand over a handwritten or typed resume, and expect a human to review it almost immediately. Today, large organizations receive thousands of applications for a single job posting, making it logistically impossible for every resume to be reviewed by a human recruiter in the first pass.

This challenge gave rise to a class of software systems known as **Applicant Tracking Systems (ATS)** — automated platforms that parse, filter, and rank resumes before they ever reach a human eye. These tools are now ubiquitous; studies consistently show that over **98% of Fortune 500 companies** and a significant majority of mid-sized firms rely on ATS platforms such as Workday, Greenhouse, Lever, Taleo, and iCIMS to manage the hiring funnel.

The consequences of this shift are profound for job seekers. A resume that would impress a human reviewer — carefully designed, visually rich, and articulately written — may be entirely invisible to an ATS if it lacks structured formatting, keyword alignment, or machine-readable text. Research conducted by career platforms suggests that **75% of resumes are rejected by ATS before a recruiter ever sees them**, not necessarily because the candidate was unqualified, but because the document was not formatted in a way the system could interpret.

1.1.2 What is an Applicant Tracking System?

An Applicant Tracking System is enterprise software used by organizations to collect, sort, scan, and rank job applications. When a candidate submits a resume online, the ATS performs several key operations:

1. **Parsing:** Extracts structured data — name, contact information, work experience, education, skills — from the raw document.
2. **Keyword Matching:** Compares the extracted content against the job description to calculate a relevance score.
3. **Ranking:** Orders candidates by score and filters out those below a cutoff threshold.
4. **Storage:** Archives parsed candidate profiles for future reference and search.

The problem arises from the inherent limitations of ATS parsers. They rely on predictable document structure, standard section headings, plain text, and common date formats. Complex visual layouts with multiple columns, embedded images, fancy fonts, headers and

footers, and HTML tables confuse most ATS parsers — causing data loss, mis-categorized entries, or outright rejection of the document.

1.1.3 The Gap Between Resume Creation Tools and ATS Compatibility

Existing resume creation tools fall into two broad categories:

Category 1 — Design-first tools (Canva, Adobe Resume Builder, Microsoft Word templates): These prioritize aesthetic appeal. They offer beautiful templates with multi-column layouts, custom fonts, and graphic elements. While visually impressive, these resumes often perform poorly in ATS environments. The multi-column layout causes ATS parsers to read table cells in the wrong order, jumbling work history and skills data. Embedded icons and profile images are often completely ignored or cause parsing errors.

Category 2 — Plain document editors (Google Docs, plain Microsoft Word, LaTeX): These produce machine-readable output but require significant manual effort. Job seekers must individually identify relevant keywords from job descriptions, rephrase bullet points using industry-standard action verbs, add quantifiable metrics and achievements, and optimize formatting — all tasks that demand knowledge of industry-specific vocabulary and professional writing conventions that most candidates simply do not possess.

Neither category addresses the full problem. There is a clear and unmet need for a **smart, AI-powered resume building tool** that simultaneously ensures ATS compatibility at the structural level and enhances content quality at the semantic level.

1.1.4 The Role of Artificial Intelligence in Recruitment Technology

Recent advances in Large Language Models (LLMs) — most notably models from the GPT series (OpenAI) and the Gemini series (Google DeepMind) — have opened transformative new possibilities for understanding and generating professional text. These models have been trained on vast corpora of professional writing, job descriptions, career advice, and resume samples. They demonstrate a sophisticated understanding of professional tone, industry-specific jargon, action verbs, STAR (Situation, Task, Action, Result) methodology, and the structural conventions of effective resumes.

Crucially, LLMs can be prompted to:

- Rewrite vague, passive bullet points into quantified, impact-driven statements
- Extract the most relevant skills from a job description and align them to a candidate's background
- Generate tailored professional summaries that position the candidate compellingly for a specific role
- Evaluate the strength and coverage of a resume relative to a target job description, returning specific, actionable improvement recommendations

This capability, combined with structured document generation and rigorous ATS-formatting rules enforced at the output layer, forms the technical foundation of the **SmartNShine ATS Resume Generator** project.

1.2 Objectives

The SmartNShine project was undertaken with the following primary and secondary objectives:

1.2.1 Primary Objectives

Objective 1 — Automated Resume Parsing

Design and implement a robust file ingestion pipeline capable of accepting resumes in multiple common formats (PDF, DOCX) and extracting their content into a structured, machine-processable JSON format. Libraries such as `pdf-parse` (for PDF) and `mammoth` (for DOCX) handle raw text extraction; the extracted text is then passed to Google Gemini via a carefully engineered prompt that instructs the model to identify and structure all standard resume sections: personal information, contact details, professional summary, work experience, education, skills, projects, certifications, and awards.

Objective 2 — AI-Powered Content Enhancement

Integrate Google Gemini and OpenAI GPT-4o APIs to provide intelligent content enhancement at the section level. Specifically:

- Transform weak, passive, or vague bullet points ("Worked on the database") into strong, action-verb-led, metric-driven statements ("Optimized PostgreSQL query execution, reducing average API response time by 45%")
- Generate or improve professional summary sections tailored to the candidate's overall experience profile
- Suggest relevant ATS keywords aligned to the candidate's target role and industry

Objective 3 — ATS-Compliant PDF Generation

Produce downloadable PDF documents that strictly adhere to ATS parsing best practices:

- Single-column layout with no tables or multi-column structures used for layout purposes
- Plain, web-safe fonts (Arial/Helvetica) for maximum compatibility
- Machine-selectable text (not rasterized image output)
- Standard, unambiguous section headings (e.g., "Experience", "Education", "Skills")
- Consistent, ATS-friendly date formats (e.g., "Month YYYY")
- Logical reading order preserved in the document object model

Objective 4 — AI Interview Practice Module

Build a comprehensive AI-powered mock interview system that allows users to:

- Configure interviews by type (technical, behavioral, HR, mixed), target role, and experience level

- Receive dynamically generated, role-relevant interview questions via GPT-4o with temperature-controlled variability
- Practice in text or voice mode, with voice transcription powered by OpenAI Whisper (STT) and text-to-speech synthesis via Chatterbox
- Receive structured, multi-dimensional feedback (relevance, clarity, confidence, technical depth, role-fit) evaluated by GPT-4o after each answer
- View a comprehensive performance report upon interview completion

Objective 5 — Secure, Authenticated Multi-User Platform

Implement a production-ready full-stack web application with:

- JWT-based session authentication for standard email and password login
- OAuth 2.0 social login via Google and GitHub (Passport.js)
- Secure per-user data isolation ensuring each user can only access their own resumes and interview sessions
- A subscription and payment system powered by Razorpay to support tiered access to premium AI capabilities
- Role-based access control (RBAC) separating regular user and administrator permissions

1.2.2 Secondary Objectives

Objective 6 — Intuitive Rich-Text Editing Experience

Provide an in-browser, WYSIWYG resume editor powered by the TipTap framework, allowing users to edit all resume sections with familiar word-processor controls (bold, italic, bullet lists) while the system transparently maintains the underlying structured data needed for AI operations and ATS-compliant export.

Objective 7 — ATS Score Analysis

Integrate an ATS score module that analyzes an uploaded or edited resume against a provided job description, returning a quantitative compatibility score (0–100) along with specific, actionable recommendations for improvement in areas such as keyword density, section completeness, and formatting compliance.

Objective 8 — Drag-and-Reorder Section Management

Allow users to reorganize resume sections — work experience entries, education entries, and project entries — through intuitive up/down controls, enabling rapid customization of the resume's emphasis for different job applications without requiring a full re-edit.

Objective 9 — Microservice Architecture for Voice AI

Design voice capabilities as independent Python Flask microservices: a Whisper STT service (port 5001) and a Chatterbox TTS service (port 5002). By decoupling voice from the main Node.js backend, the architecture enables independent scaling per service,

technology stack flexibility, and fault isolation — a voice service failure does not cascade to the rest of the application.

Objective 10 — Comprehensive Admin Dashboard

Develop an administrative interface providing operational visibility into user accounts, AI usage statistics (tracked per call via the `AIUsage` model), payment records, and system health metrics, accessible exclusively to users with the `admin` role enforced at the middleware level.

1.3 Purpose of the Project

1.3.1 Solving a Real-World Problem

The SmartNShine ATS Resume Generator was conceived as a practical solution to a well-documented problem that affects millions of job seekers globally every year. The purpose of this project extends beyond academic exercise; it targets a genuine pain point in the recruitment ecosystem and is designed to be commercially viable.

Consider the experience of a recent engineering graduate attempting to enter the workforce. They may possess excellent technical skills, a solid academic record, and genuine hands-on project experience. However, they may lack the professional writing experience to articulate their capabilities in resume language optimized for ATS systems. They may not know that "Worked on the database" is significantly weaker — and less ATS-visible — than "Optimized PostgreSQL query performance, reducing API response time from 800ms to 440ms." They may not know that a visually polished three-column Canva template will be silently rejected by Workday long before a human recruiter ever reads it.

SmartNShine addresses every layer of this problem systematically:

- **Structural compliance:** ATS-safe, single-column PDF output with machine-readable text
- **Content intelligence:** Gemini/GPT-4o rewrites weak bullet points into high-impact, measurable statements
- **Keyword alignment:** AI-suggested keywords matched to specific job descriptions
- **Interview preparation:** AI-driven simulated interviews that build communication confidence and surface improvement areas

1.3.2 Demonstrating the Integration of Modern AI into Full-Stack Applications

From a technical standpoint, this project serves as a concrete demonstration of how **Large Language Models can be integrated as functional services within a production-grade, full-stack web application**. The project is not a simple chatbot interface layered over a model API; it is a complete software product in which AI serves as a strategic component within a broader, architecturally sound system — one that includes authentication, payments, persistent data storage, real-time UI updates, and voice pipelines.

Key technical demonstrations include:

- **Prompt engineering:** Carefully constructed prompts with structured output requirements, few-shot examples, and chain-of-thought instructions guide the LLM to produce JSON-formatted responses suitable for direct database insertion and UI rendering
- **Multi-model routing:** The system intelligently routes tasks to the most appropriate AI backend — Google Gemini for resume parsing and content enhancement (fast, cost-efficient, strong at structured extraction) and OpenAI GPT-4o for interview question generation and answer evaluation (nuanced reasoning, multi-dimensional scoring)
- **Voice AI pipeline:** End-to-end voice interaction using Whisper for high-accuracy transcription and Chatterbox for natural speech synthesis, integrated into a conversational interview session with real-time audio processing

1.3.3 Internship Learning Outcomes

This project was developed over the course of an industry internship, with the explicit goal of providing hands-on, production-caliber exposure to challenges encountered in real software engineering teams. Five distinct sub-projects were completed and integrated during this period:

1. **ATS Resume Builder Core** — Full resume lifecycle from PDF/DOCX upload, through AI-powered parsing, WYSIWYG editing via TipTap, to ATS-optimized PDF export via `react-to-print`
2. **AI Content Enhancement Engine** — Gemini-powered bullet point enhancement, professional summary generation, and resume content scoring
3. **ATS Score Analyzer** — Resume-to-job-description compatibility analysis returning a numeric score and ranked improvement recommendations
4. **AI Mock Interview System** — GPT-4o-driven interview sessions with configurable parameters, real-time answer evaluation across five dimensions, and comprehensive final reports stored in MongoDB
5. **Voice Interview Pipeline** — OpenAI Whisper STT microservice and Chatterbox TTS microservice integrated into the interview UI, enabling fully voice-conducted practice interviews

Each sub-project demanded independent research, architecture design, implementation, integration testing, and documentation — providing comprehensive, lifecycle-spanning experience.

1.3.4 Target Users

The platform is designed to serve the following user segments:

User Segment	Primary Need Addressed
Final-year students / Fresh graduates	First-time resume creation with professional-quality, ATS-ready output
Mid-career professionals	Resume refresh, ATS optimization, and keyword alignment for role transitions
Interview candidates	Structured mock interview practice with detailed AI feedback
Career coaches and mentors	A recommended platform for guiding clients through resume and interview preparation
Platform administrators	Operational monitoring, user management, AI usage tracking, and payment records

1.3.5 Broader Impact and Democratization of Career Tools

Beyond direct utility to individual users, this project contributes to a broader conversation about **access and equity in the recruitment process**. AI-powered resume optimization — keyword tailoring, professional tone enhancement, ATS formatting — has historically been available only to candidates who could afford professional resume writing services (typically costing ₹5,000–₹40,000 per resume). SmartNShine makes these capabilities accessible to any candidate with internet access, leveling the playing field and reducing the structural advantages that financial resources and professional networks provide in the hiring process.

Furthermore, by designing the platform transparently — showing users *what* makes a resume ATS-friendly and *why* — the application builds informed, self-sufficient job seekers rather than creating a black-box dependency on automation.

1.4 Scope of the Report

This technical report documents the complete design, implementation, evaluation, and conclusion of the SmartNShine ATS Resume Generator project across six chapters:

Chapter	Title	Contents
Chapter 1	Introduction	Background, objectives, purpose, project scope
Chapter 2	Literature Survey	Review of ATS systems, AI writing tools, voice pipelines, prior art
Chapter 3	System Design	High-level architecture, DFDs, component design, design rationale
Chapter 4	Implementation	Tech stack, frontend and backend code, AI integration, prompt engineering
Chapter 5	Results and Evaluation	Sample outputs, performance metrics, future scope, internship summary
Chapter 6	Conclusion	Accomplishments, learnings, and closing remarks

1.5 Summary

This chapter established the context and motivation for the SmartNShine ATS Resume Generator project. The modern hiring landscape, dominated by Applicant Tracking Systems, creates systematic barriers for job seekers whose resumes are eliminated before human review — not due to a lack of qualifications, but due to structural and semantic incompatibilities that most candidates are entirely unaware of. Existing tools either prioritize visual design at the expense of machine readability, or require candidates to perform complex optimization tasks that demand expertise they do not possess.

SmartNShine addresses this gap through a complete full-stack MERN web application that integrates Google Gemini and OpenAI GPT-4o for intelligent resume parsing, content enhancement, ATS compatibility scoring, and AI-driven interview practice — complemented by a voice interview pipeline built on OpenAI Whisper and Chatterbox. The project's objectives span user-facing functionality, architectural soundness, and security — targeting a commercially viable product while serving simultaneously as a comprehensive learning platform for modern, AI-integrated full-stack software development.

End of Chapter 1

Chapter 2: Literature Survey

2.1 Introduction to the Literature Survey

A literature survey is the systematic review of published research, technical documentation, prior systems, and industry reports relevant to the problem domain being addressed. For the SmartNShine ATS Resume Generator, the literature spans four intersecting domains: (1) Applicant Tracking Systems and their role in recruitment automation, (2) Natural Language Processing and Large Language Models as applied to professional text, (3) Voice Artificial Intelligence — both speech-to-text and text-to-speech systems, and (4) full-stack web engineering patterns for AI-integrated applications.

This chapter reviews foundational and contemporary work in each of these areas, identifies the gaps that motivate SmartNShine's design choices, and situates the project within the current state of related commercial and academic tools.

2.2 Applicant Tracking Systems: History and Mechanics

2.2.1 Origins and Evolution

The concept of automated candidate tracking originated in the 1990s as organizations began digitizing their HR operations. Early systems, such as Resumix (acquired by Yahoo in 2004), performed basic keyword matching using Boolean logic on plain-text resumes. The systems were rudimentary — they identified the presence or absence of exact keyword strings and ranked candidates accordingly.

By the 2000s, as internet job boards such as Monster.com and HotJobs proliferated, the volume of applications per posting exploded. Companies began receiving hundreds to thousands of applications for individual roles, making even first-pass manual screening economically unsustainable. This drove adoption of more sophisticated ATS platforms — Taleo (founded 1999, acquired by Oracle 2012), Kenexa BrassRing (acquired by IBM 2012), and later Workday (HR module launched ~2012), Greenhouse (2012), and Lever (2012).

Modern ATS platforms combine:

- **Resume parsing engines** to extract structured data
- **Keyword and semantic matching** against job descriptions
- **Workflow management** tools for recruiter collaboration
- **Communication automation** for candidate outreach
- **Analytics dashboards** for pipeline reporting

2.2.2 How Modern ATS Parsers Work

Academic research (Celik & Elci, 2013; Kopparapu, 2010) on information extraction from resumes identifies the primary technical challenges as: section segmentation, entity recognition, and relationship mapping.

Section Segmentation: The parser must identify boundaries between resume sections — where the "Experience" section ends and "Education" begins. Most parsers rely on a combination of heuristic rules (e.g., detecting heading-like text following whitespace) and statistical models trained on labeled resume corpora.

Named Entity Recognition (NER): Within sections, the parser extracts entities such as person names, company names, educational institutions, degrees, dates, and skill terms. Early approaches used hand-crafted rule sets; modern parsers use supervised NER models (Conditional Random Fields, BiLSTM-CRF).

Relationship Mapping: Correctly associating a job title with a specific company and date range is non-trivial when formatting is inconsistent. Parsers may incorrectly map a job title from one employer to another when layout conventions don't match the expected pattern.

The critical insight from this literature is that **formatting consistency is as important as content quality** for reliable ATS parsing. This directly motivated SmartNShine's commitment to single-column, semantically structured PDF output.

2.2.3 The ATS Filtering Problem

Research published by the Harvard Business School (Fuller et al., 2021), "Hidden Workers: Untapped Talent," found that ATS systems contribute to systematic talent mismatches. Their study of over 2,500 employers found that rigid ATS keyword filters cause organizations to automatically screen out millions of qualified candidates — the so-called "hidden workers" — who lack specific keyword phrasing even when they possess the underlying competencies.

Key findings relevant to SmartNShine's design:

- **Keyword phrasing matters more than underlying competency:** A candidate who wrote "built web apps" may be rejected while one who wrote "developed React single-page applications" passes, even if both have equivalent skills.
- **Format-sensitive parsers compound the problem:** Candidates using visually rich templates suffer parsing failures independent of their qualifications.
- **75% rejection rate before human review:** The majority of candidates never receive human evaluation of their application.

These findings confirm the core value proposition of an AI-powered tool that both optimizes keyword alignment and enforces ATS-compliant formatting at the point of export.

2.3 Natural Language Processing for Resume Understanding

2.3.1 Early NLP Approaches to Resume Parsing

Early academic work on automated resume processing framed the problem as a sequence labeling task. Yu et al. (2005) presented one of the first systematic approaches, using a cascaded information extraction framework that segmented resumes into sections and applied NER within each segment. Their system achieved ~80% field extraction accuracy on a controlled dataset.

Subsequent work explored machine learning approaches. Celik & Elci (2013) applied Support Vector Machines (SVMs) to classify resume lines as belonging to specific section types. Luo et al. (2014) used Conditional Random Fields (CRFs) for joint section segmentation and entity extraction, achieving state-of-the-art results at the time.

The limitations of these systems were significant:

- Heavy reliance on training data with consistent formatting
- Poor generalization to novel resume styles
- No understanding of semantic meaning — only pattern matching
- Inability to handle acronym variants ("JS" vs. "JavaScript")

2.3.2 The Transformer Revolution and Its Impact on Resume Processing

The publication of "Attention Is All You Need" (Vaswani et al., 2017) introduced the Transformer architecture, which subsequently became the foundation for a generation of massively powerful language models. BERT (Devlin et al., 2018), GPT-2 (Radford et al., 2019), GPT-3 (Brown et al., 2020), and their successors demonstrated emergent capabilities in text understanding, generation, and instruction following that qualitatively exceeded all prior approaches.

For resume processing, transformer-based models offered dramatic improvements:

- **Semantic understanding:** Unlike keyword matching, transformers understand that "Python" and "programming in Python" are related, and that "led a team" implies leadership skills even without the word "leadership"
- **Zero-shot and few-shot generalization:** Models like GPT-3 could follow natural language instructions for extraction tasks without task-specific fine-tuning
- **Multi-section context:** Attention mechanisms allow the model to resolve ambiguities using document-wide context

The commercial resume parsing API market responded rapidly. HireAbility, Sovren (now Textkernel), and Rchilli all transitioned to transformer-enhanced parsing pipelines between 2019 and 2022.

2.3.3 Google Gemini and Its Application in SmartNShine

Google Gemini (launched December 2023) represents Google DeepMind's flagship multimodal language model series. The `gemini-2.5-flash` variant used in SmartNShine is optimized for high-throughput, cost-efficient text tasks with strong instruction-following. Its key characteristics relevant to this project:

Structured Output Reliability: Gemini demonstrates consistent ability to produce valid JSON output when instructed. This is critical for SmartNShine's parsing pipeline, which passes raw extracted resume text and expects a structured JSON object back — directly insertable into MongoDB without intermediate transformation.

Long Context Window: Gemini 2.5 Flash supports up to 1 million tokens of context. While resumes are short documents, this capacity ensures that full job descriptions and complete resume text can be processed simultaneously during ATS score analysis.

Instruction Following Fidelity: Prompts with complex, multi-rule constraints (such as SmartNShine's ENHANCE_CONTENT_PROMPT, which specifies strict word limits per bullet, experience-level-aware writing, and prohibition on fabricating metrics) are followed with high reliability.

The decision to use Gemini for parsing and content enhancement (rather than OpenAI) reflects both API cost considerations and observed performance: Gemini's flash models offer faster latency and lower per-token cost for structured extraction tasks, while GPT-4o's stronger reasoning capabilities are reserved for nuanced interview evaluation.

2.3.4 OpenAI GPT-4o and Interview Evaluation

GPT-4o (OpenAI, May 2024) is a multimodal "omni" model representing OpenAI's state-of-the-art for reasoning, instruction following, and nuanced language understanding. Its relevance to SmartNShine's interview module stems from several properties:

Multi-dimensional Scoring: GPT-4o demonstrates strong capability for structured evaluation tasks. SmartNShine uses it to simultaneously score interview answers on five dimensions — relevance, technical accuracy, clarity, confidence, and role-fit — returning a single structured JSON object with all scores, strengths, weaknesses, and a suggested ideal answer.

Adaptive Question Generation: With temperature set to 0.7, GPT-4o generates interview questions with controlled variability — consistent enough to be professional, varied enough to avoid repetition across sessions. The system prompt encodes interviewer persona, experience-level expectations, and question diversity requirements.

Reasoning Quality: Complex behavioral interview evaluation — assessing whether a STAR-format answer adequately demonstrates the competency being tested — requires the kind of deep reasoning that smaller models handle poorly. GPT-4o's performance on such tasks is consistently superior to lighter alternatives.

2.4 Prompt Engineering: Techniques and Best Practices

2.4.1 Definition and Emergence

Prompt engineering emerged as a recognized discipline alongside the rise of few-shot capable LLMs. Unlike traditional software parameters, prompts are natural language instructions that shape model behavior. The field consolidates findings from the LLM research community and practitioners who discovered that systematic prompt design dramatically affects output quality, reliability, and format adherence.

Wei et al. (2022) formally described **chain-of-thought prompting** — the technique of including reasoning steps in the prompt to elicit more thorough and accurate responses. Brown et al. (2020) demonstrated **few-shot prompting** — providing example input-output pairs within the prompt — as a powerful technique for guiding output format and style without fine-tuning.

2.4.2 Techniques Employed in SmartNShine

SmartNShine's AI service layer implements several prompt engineering strategies worth noting in this literature context:

Structured Output Enforcement: All Gemini and GPT-4o prompts include explicit JSON schema specifications. The instruction "Return ONLY valid JSON with no additional text, explanations, or markdown formatting" consistently elicits clean parseable output. The application then strips any residual markdown code fences (````json`) before parsing.

Few-Shot Examples: The ATS score prompt includes a complete example response structure showing the exact field names, data types, and value ranges expected. This acts as an implicit few-shot example, anchoring the model's output format.

Role-Persona Priming: Both the interviewer and evaluator system prompts begin with a role definition ("You are an experienced technical interviewer...", "You are an expert interview evaluator..."). Research by White et al. (2023) in "A Prompt Pattern Catalog" confirms that persona priming improves domain-relevant output consistency.

Constraint Enumeration: SmartNShine's content enhancement prompt enumerates 6 critical rules with explicit length limits, prohibition on fabricating metrics, experience-level awareness, and formatting conventions. Enumerating constraints with numbered lists and **bold headings** has been shown to improve constraint adherence compared to prose-format instructions (Liu et al., 2023).

Context Injection: The enhancement prompt injects a computed `resumeContext` block containing the candidate's experience level classification (FRESHER / JUNIOR / SENIOR), total years of experience, and skill count. This context prevents the model from "hallucinating" senior-level achievements into a fresh graduate's resume — a critical content integrity safeguard.

2.4.3 Retry Logic and Reliability Engineering

Production LLM integrations must contend with API instability — rate limits (HTTP 429), service unavailability (HTTP 503), and transient server errors. SmartNShine implements an exponential backoff retry strategy with jitter:

```
delay = min(baseDelay * 2^attempt + random(0, 1000ms), 10000ms)
```

This pattern — described in "Exponential Backoff and Jitter" (Amazon Web Services, 2015) — prevents thundering herd problems when multiple concurrent requests encounter a rate limit. SmartNShine's implementation performs up to 3 retry attempts for retryable error codes (429, 500, 502, 503, 504) before propagating the error to the client.

2.5 Voice Artificial Intelligence

2.5.1 Automatic Speech Recognition (ASR): Background

Automatic Speech Recognition — converting spoken audio to text — has a long research history beginning with rule-based phoneme recognition systems in the 1950s. The field progressed through Hidden Markov Models (HMMs) in the 1980s, hybrid HMM-DNN systems in the 2010s, and ultimately end-to-end deep learning architectures in the late 2010s.

The dominant paradigm shift came with transformer-based end-to-end ASR. Models such as DeepSpeech 2 (Baidu, 2016) and later wav2vec 2.0 (Facebook AI Research, 2020) demonstrated that large self-supervised pre-training on audio data followed by fine-tuning on labeled speech could achieve competitive performance with substantially less labeled data than HMM-DNN systems.

2.5.2 OpenAI Whisper

OpenAI Whisper (Radford et al., 2022) is an open-source ASR model trained on 680,000 hours of multilingual, multitask weakly supervised audio data collected from the internet. Its key properties that drove SmartNShine's STT selection:

Robustness: Whisper demonstrates exceptional robustness to noise, accents, and recording conditions compared to prior open-source alternatives. For an interview practice system where users may be recording from laptops in varied acoustic environments, this robustness is essential.

Multilingual Support: Whisper supports 99 languages out of the box. While SmartNShine currently operates in English, multilingual support is a meaningful future-scope item for non-English-speaking job markets.

Open-Source Availability: Unlike cloud ASR APIs (Google Speech-to-Text, Amazon Transcribe, Azure Speech), Whisper can be self-hosted, providing cost predictability and data privacy — interview recordings do not leave the infrastructure.

Architecture: Whisper uses a standard transformer encoder-decoder architecture. The encoder processes log-Mel spectrogram features extracted from audio; the decoder autoregressively generates transcribed text tokens. The `base` and `small` model variants are used for low-latency scenarios; SmartNShine's voice-service/app.py can be configured for any variant based on deployment hardware.

SmartNShine deploys Whisper as a standalone Flask microservice on port 5001. The Node.js backend acts as a secure proxy, forwarding audio blobs from the frontend to the voice service and returning transcription text — ensuring the voice service is never directly exposed to the browser.

2.5.3 Text-to-Speech Synthesis: Background and Chatterbox

Text-to-Speech (TTS) synthesis — converting written text to natural-sounding speech — has similarly evolved from rule-based concatenative systems to neural end-to-end models.

Concatenative TTS (1990s–2000s): assembled pre-recorded phoneme segments. Natural-sounding in specific conditions but robotic in variability.

Statistical Parametric TTS (2000s–2010s): HMM-based systems that generated acoustic parameters from text, synthesized via vocoders. More flexible but with characteristic "buzzy" quality.

Neural TTS (2016–present): WaveNet (van den Oord et al., 2016) introduced autoregressive waveform synthesis, producing near-human-quality speech at the cost of high inference latency. Tacotron (Wang et al., 2017) and Tacotron 2 combined a sequence-to-sequence model for spectrogram generation with a WaveNet vocoder. FastSpeech and FastSpeech 2 (Ren et al., 2019, 2020) achieved non-autoregressive synthesis, dramatically improving inference speed without sacrificing quality.

Chatterbox TTS is the open-source TTS system selected for SmartNShine's voice interview pipeline. Key characteristics:

- **Pure Python, self-hosted:** No external API call; synthesis runs locally within the microservice, ensuring privacy and zero per-character cost
- **Neural quality:** Uses a modern neural vocoder architecture producing naturalistic speech suitable for interview question delivery
- **Configurable voice parameters:** Sample rate, voice style, and speed are adjustable through the Flask API
- **Low latency:** Optimized for real-time use — the TTS service must synthesize question audio quickly enough to maintain conversational pacing in the interview

SmartNShine initially integrated ElevenLabs TTS (a commercial API offering highly realistic voice synthesis). However, due to payment and quota issues, ElevenLabs was disabled and all TTS endpoints return HTTP 503 with a descriptive error. Chatterbox was selected as the replacement due to its open-source nature and self-hosted deployment model. The browser's native Web Speech API serves as a second-level fallback if Chatterbox is unavailable.

2.5.4 Voice Pipeline Architecture in Research Context

The SmartNShine voice pipeline implements a priority chain consistent with best practices described in conversational AI literature (McTear, 2016; Skantze, 2021):

```
TTS Priority: Chatterbox (port 5002) → Browser Web Speech API  
STT Priority: Whisper (port 5001) via server proxy
```

This cascade design ensures service continuity — if the high-quality Chatterbox service is unavailable, the interview continues with browser TTS. The degradation is graceful: interview functionality is preserved even if the microservices are down, at the expense of voice quality.

2.6 Review of Related Systems and Commercial Tools

2.6.1 Resume Enhancement Tools

Jobscan (jobscan.co): A commercial tool that analyzes a resume against a specific job description and provides a keyword match score. Jobscan pioneered the concept of ATS score visualization for job seekers. Its limitation is that it is a purely analytical tool — it identifies gaps but does not automatically fix them. The user must manually rewrite their resume based on recommendations.

SmartNShine's differentiator: Automated AI rewriting that transforms identified gaps into enhanced content without requiring manual editing skill.

Resumake / Resume.io / Zety: Template-based resume builders that prioritize visual design. Most offer step-by-step form-filling interfaces and produce PDF exports. They do not offer AI content enhancement or ATS scoring in their core free tiers.

SmartNShine's differentiator: AI-powered content optimization and ATS compliance checking integrated into the editing workflow, not as an afterthought.

Kickresume / Enhancv: Premium resume builders with AI-assisted content suggestions (typically powered by GPT). These tools represent the closest commercial analogs to SmartNShine's resume module.

SmartNShine's differentiator: Full-stack open-source implementation with document upload and parse (users can start from their existing resume, not from a blank form), plus the AI Interview module — a feature none of the above tools offer.

Rezi.ai: An AI resume builder specifically marketed for ATS optimization. Uses GPT to generate content and enforces single-column templates. Strong ATS focus.

SmartNShine's differentiator: End-to-end voice interview practice, open-source architecture, and Razorpay-based payment integration.

2.6.2 AI Interview Practice Tools

Interviewing.io: A platform for practicing technical interviews with anonymous peer engineers. Human-to-human, not AI-driven. High quality but expensive and supply-constrained.

Pramp: Peer-to-peer mock interview platform. Again human-based; scheduling-dependent.

HireVue: An enterprise video interview platform that uses AI to analyze candidate responses for sentiment, vocabulary, and non-verbal cues. Used by employers as a screening tool, not a candidate preparation tool.

Google Interview Warmup: A free, lightweight AI interview practice tool. Provides pre-set question banks for specific roles and evaluates answers for keywords and talking points. However, it does not offer multi-dimensional scoring, personalized feedback, voice synthesis for questions, or integration with a candidate's actual resume.

Yoodli: An AI speech coaching tool that evaluates communication patterns (pacing, filler words, conciseness) in interview-style responses. Focused on communication delivery rather than technical content.

SmartNShine's differentiator: End-to-end interview session with dynamic GPT-4o question generation (adaptive difficulty, no fixed question banks), five-dimension answer evaluation, comprehensive reporting, voice question delivery via Chatterbox TTS, and direct integration with the user's own resume data.

2.6.3 Full-Stack MERN Application Patterns

The MERN stack (MongoDB, Express.js, React, Node.js) has become a dominant paradigm for full-stack JavaScript web applications. Its adoption is well-documented in the developer community (StackOverflow Developer Survey, 2023 — MongoDB and Node.js consistently rank among the top-5 databases and back-end technologies).

Key architectural decisions in SmartNShine align with established MERN best practices:

MVC + Service Layer: The backend separates concerns into controllers (HTTP request handling), services (business logic, AI calls), models (MongoDB schema definitions), and routes (URL pattern registration). This separation facilitates unit testing and makes the AI services independently testable.

JWT Authentication: JSON Web Tokens (JWTs) for stateless session management is a widely adopted pattern for RESTful APIs (Jones et al., RFC 7519, 2015). SmartNShine uses 7-day expiring JWTs, consistent with industry norms.

Social OAuth via Passport.js: Passport.js (passportjs.org) is the de-facto Node.js authentication middleware. Its strategy-based architecture allows SmartNShine to support Google and GitHub OAuth alongside local email/password authentication with minimal code duplication.

Microservice Architecture for Python Services: Node.js is not the natural home for ML workloads — Python's ecosystem (PyTorch, Hugging Face, Whisper) is far richer. The decision to implement voice services as independent Python Flask microservices, communicating with the Node.js backend via HTTP, reflects the principle of polyglot microservices: use the right language for the right job (Newman, 2015, "Building Microservices").

2.7 Rich Text Editing: TipTap

2.7.1 The WYSIWYG Editor Landscape

Web-based rich text editors have evolved significantly. Legacy approaches (TinyMCE, CKEditor) relied on `contenteditable` with significant browser inconsistency handling overhead. ProseMirror (Haverbeke, 2015) introduced a principled document model that treats the editor state as an immutable data structure — ensuring predictable, testable behavior.

TipTap is a headless, framework-agnostic wrapper built on top of ProseMirror, with a React integration layer. "Headless" means it ships no default styling — the consuming application owns the visual appearance entirely. This is ideal for SmartNShine, where the editor's output must integrate with TailwindCSS-styled components.

2.7.2 TipTap in SmartNShine

SmartNShine uses TipTap's `@tiptap/react` integration with the `@tiptap/starter-kit` extension bundle. The editor is embedded within the `EditableSection` component, rendering each resume section (experience bullets, project descriptions, summary) as an independently editable TipTap instance.

Key design decision: TipTap's editor state produces HTML output (via `editor.getHTML()`). SmartNShine's data layer stores and processes resume content as structured JSON arrays of strings. The application layer therefore maintains bidirectional conversion between TipTap HTML and structured bullet arrays, ensuring that AI enhancement operates on clean text data while the editor presents rich formatted text to the user.

2.8 Document Generation and ATS-Compliant PDF Export

2.8.1 PDF Generation Approaches in Web Applications

Web-based PDF generation falls into two categories:

Server-side generation: Libraries such as `pdfkit` (Node.js), `reportlab` (Python), or `wkhtmltopdf` (HTML to PDF conversion) generate PDFs on the server and stream them to the client. Server-side generation offers precise control over layout but requires additional server resources and introduces a round-trip delay.

Client-side generation: The browser's print API (`window.print()`), `jsPDF` , or `react-to-print` can generate PDFs from DOM elements entirely in the browser. Client-side generation is faster for the user (no server round-trip) and reduces server load, but relies on browser rendering — which can produce inconsistent output across browsers.

2.8.2 SmartNShine's Approach: react-to-print

SmartNShine uses `react-to-print` — a library that triggers the browser's native print dialog targeting a specific DOM element (the resume preview component). The user's browser renders the `ResumePreview` component to PDF.

This approach has a direct implication for ATS compliance: the PDF's text layer is produced by the browser's PDF renderer from actual DOM text nodes. Unlike image-based PDF export, the text is fully selectable and machine-readable — a non-negotiable requirement for ATS parsing.

The `ResumePreview` component is designed with strict ATS-compliance:

- No multi-column CSS grid or flexbox layouts
- No images or icons (ATS parsers cannot read them)
- Standard HTML heading elements (`<h1>` , `<h2>` , `<h3>`) for section structure
- Web-safe fonts (Arial) specified in inline styles
- Consistent `<ul>` / `<li>` bullet lists for experience entries

This design aligns with Jobscan's published ATS formatting guidelines and practices documented by major resume coach communities.

2.9 Payment Integration: Razorpay

2.9.1 Background on Online Payment Gateways

Payment gateway integration is a standard component of SaaS applications with premium tiers. Options for Indian-market applications include Razorpay, PayU, CCAvenue, and Cashfree. For international applications, Stripe dominates.

Razorpay is the leading full-stack payments platform for Indian businesses, processing transactions in INR and supporting UPI, net banking, credit/debit cards, and wallets. Its developer API implements a two-phase payment pattern:

1. **Order Creation (server-side):** The backend creates a Razorpay order with amount and currency, receiving an `order_id`
2. **Payment Collection (client-side):** The frontend loads the Razorpay checkout widget with the `order_id` ; the user completes payment
3. **Verification (server-side):** The backend verifies the payment signature using HMAC-SHA256 to confirm the transaction's authenticity before unlocking premium features

2.9.2 Razorpay in SmartNShine

SmartNShine integrates Razorpay via the `payment.service.js` service layer. The subscription model gates access to unlimited AI enhancements and the full interview module. The `Subscription` MongoDB model tracks plan type, payment reference, and expiration date per user. The HMAC signature verification step is critical for security — it prevents clients from forging successful payment responses.

2.10 Summary of Literature Survey

This chapter has reviewed the relevant literature and prior work across the five primary domains of the SmartNShine project. The key takeaways are:

Domain	Key Finding	SmartNShine's Response
ATS Systems	75% of resumes rejected before human review; formatting and keyword gaps are the primary causes	Enforced ATS-compliant single-column PDF; AI-powered keyword alignment
LLM-based NLP	GPT-4o and Gemini 2.5 Flash offer state-of-the-art structured extraction and generation at production scale	Gemini for parsing/enhancement; GPT-4o for interview evaluation
Prompt Engineering	Structured constraints, persona priming, and few-shot examples dramatically improve LLM output reliability	Multi-rule prompts with JSON schema, experience-level context injection, retry with exponential backoff
Voice AI	Whisper excels at robust, open-source STT; neural TTS (Chatterbox) enables self-hosted voice synthesis	Whisper STT microservice + Chatterbox TTS microservice with browser fallback
Related Tools	No existing tool combines ATS optimization, AI content rewriting, voice interview practice, and payment-gated subscription in a single open-source platform	SmartNShine's complete feature set is uniquely positioned in this space

The gap analysis confirms that SmartNShine occupies a genuinely differentiated position in the current tool landscape — one that this project's implementation validates through end-to-end technical construction.

End of Chapter 2

Chapter 3: System Design

3.1 Overview

System design is the process of defining the architecture, components, interfaces, and data flows of a system to satisfy specified requirements. For SmartNShine, the design must satisfy a diverse set of simultaneous requirements: real-time AI-powered text processing, multi-format document ingestion, voice-based interaction, secure multi-user data isolation, payment integration, and ATS-compliant PDF generation — all within a responsive web application.

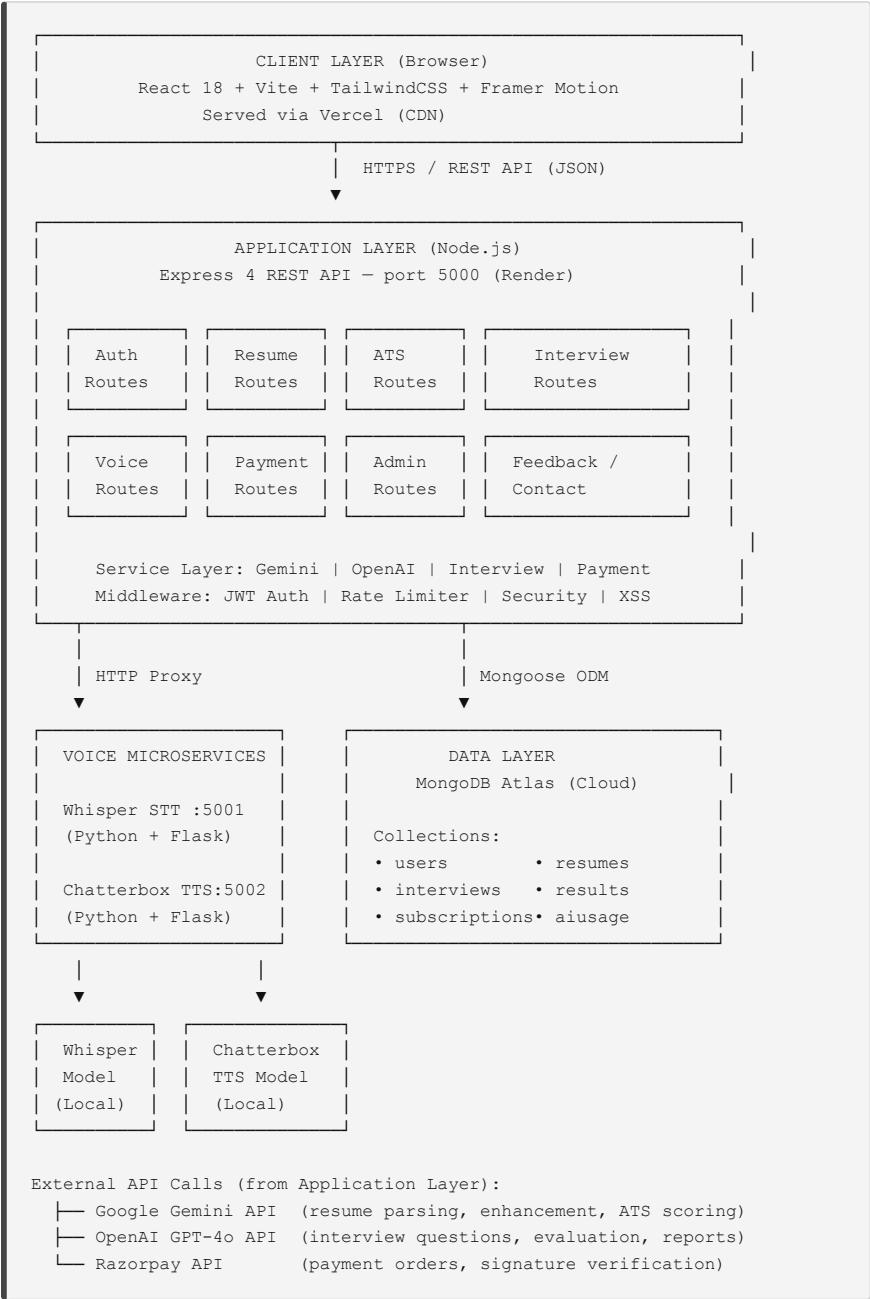
This chapter presents the complete system design across multiple levels of abstraction: high-level architecture, component-wise design, data flow diagrams, and database schema design. Where relevant, specific design decisions and their rationale are explained.

3.2 High-Level Architecture

3.2.1 Architectural Style

SmartNShine is designed as a **three-tier web application** with an additional **microservice layer** for AI voice processing:

- **Tier 1 — Presentation Layer:** React 18 + Vite frontend, served via Vercel
- **Tier 2 — Application Layer:** Node.js + Express 4 REST API backend, deployed on Render
- **Tier 3 — Data Layer:** MongoDB Atlas cloud database
- **Microservice Layer:** Two independent Python Flask servers for voice AI (Whisper STT, Chatterbox TTS)
- **External Services:** Google Gemini API, OpenAI API, Razorpay, nodemailer (SMTP)

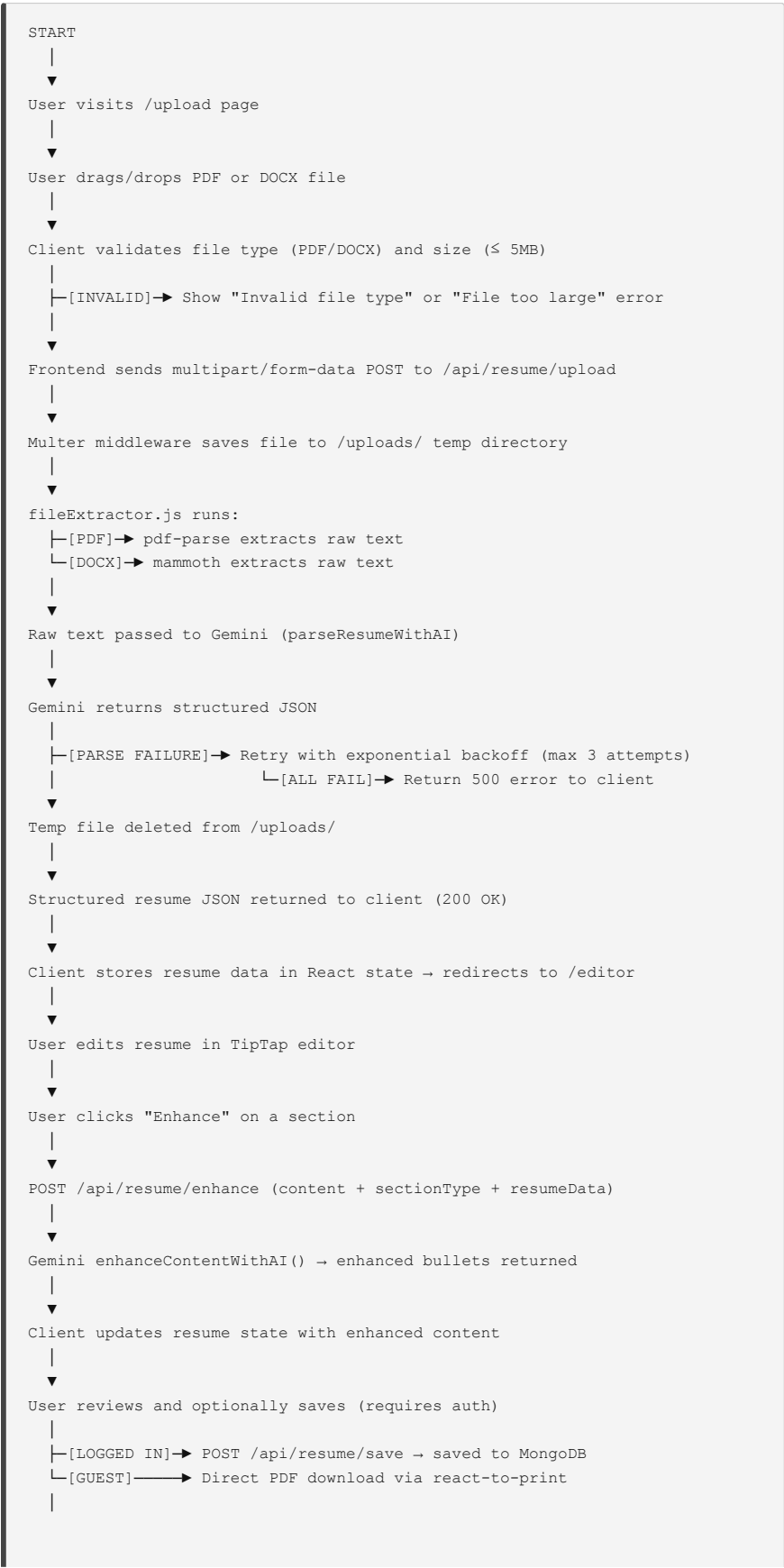


3.2.2 Communication Protocols

Connection	Protocol	Format
Browser ↔ Node.js Backend	HTTPS + REST	JSON
Node.js ↔ MongoDB Atlas	MongoDB Wire Protocol (TCP)	BSON
Node.js ↔ Whisper microservice	HTTP (internal)	multipart/form-data
Node.js ↔ Chatterbox microservice	HTTP (internal)	JSON / audio stream
Node.js ↔ Gemini API	HTTPS	JSON
Node.js ↔ OpenAI API	HTTPS	JSON
Node.js ↔ Razorpay API	HTTPS	JSON
Browser ↔ Razorpay Checkout	HTTPS	Razorpay SDK

3.3 Flowchart: Core User Workflows

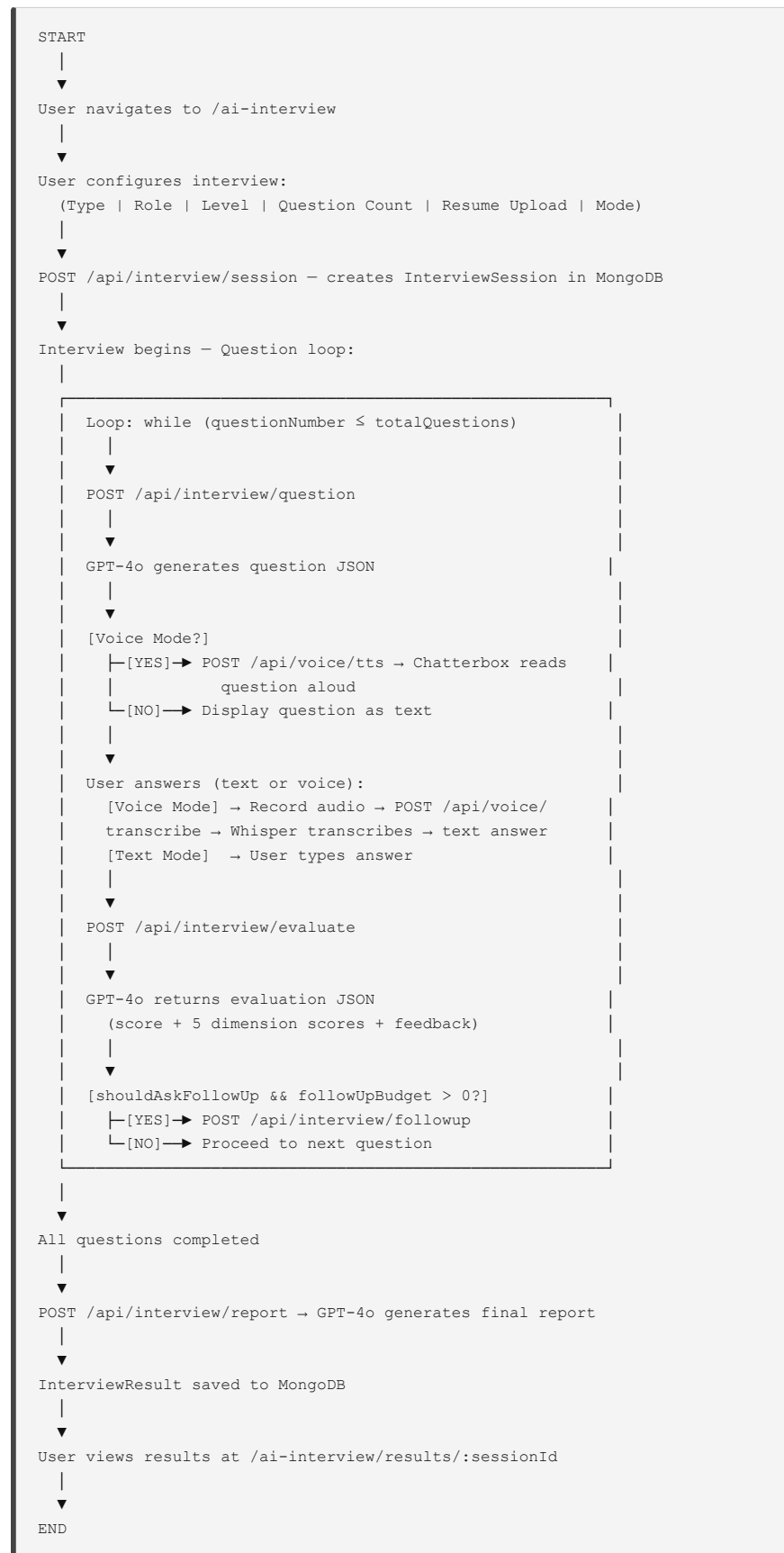
3.3.1 Resume Upload and AI Enhancement Workflow



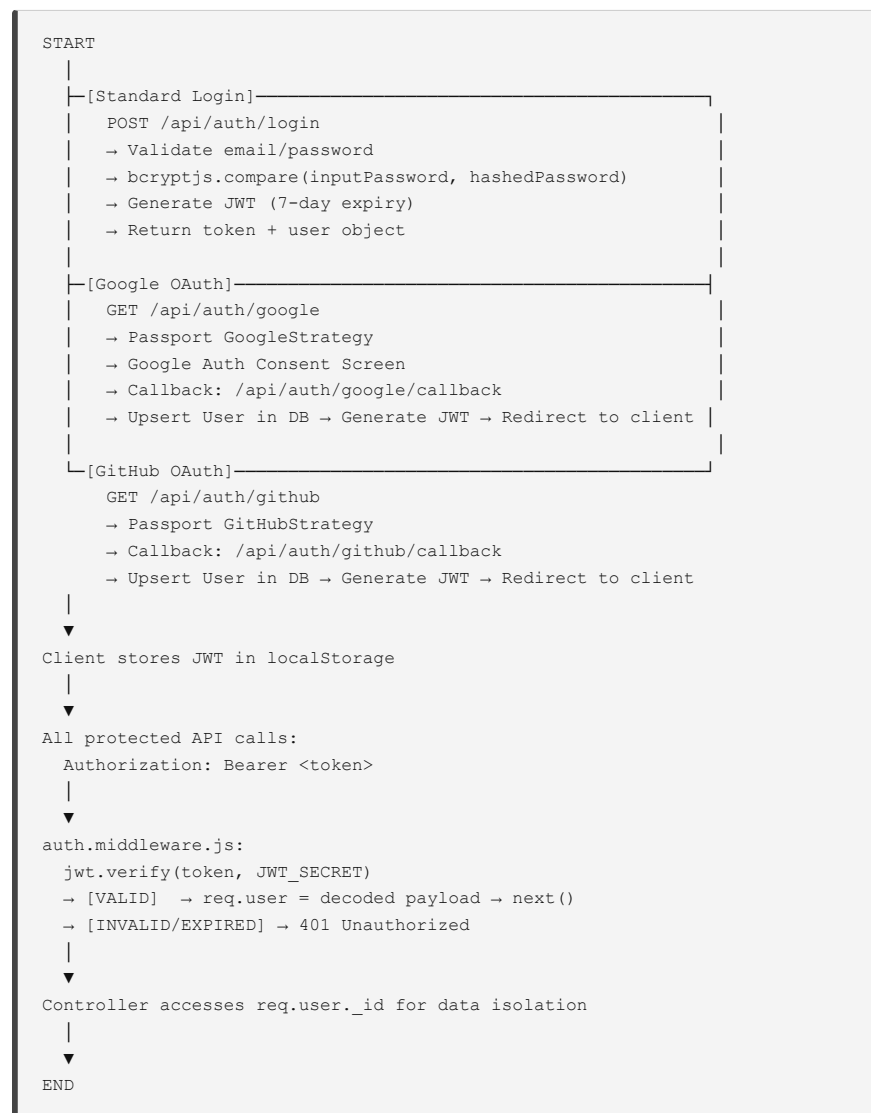
▼

END

3.3.2 AI Interview Workflow



3.3.3 Authentication and Authorization Workflow



3.4 High-Level Application Architecture

3.4.1 Frontend Architecture (React + Vite)

The frontend is organized into a domain-driven structure:

```

client/src/
├── api/                # Axios API clients
│   ├── api.js         # Resume, auth, ATS API calls
│   └── interview.api.js # Interview session API calls
├── components/         # Reusable UI components
│   ├── admin/          # Admin dashboard widgets
│   ├── auth/           # Login, Register, OAuth buttons
│   ├── common/         # Navbar, Footer, LoadingSpinner
│   ├── editor/         # TipTap editor wrappers
│   ├── layout/         # BaseLayout, PageWrapper
│   └── templates/      # Resume preview templates
├── context/           # React Context providers
│   ├── DarkModeContext.js # Global dark mode state
│   └── NavigationBlockerContext.js # Unsaved changes guard
├── hooks/             # Custom React hooks
├── pages/             # Route-level page components
│   ├── Home.jsx
│   ├── Upload.jsx
│   ├── Editor.jsx      # Main resume editor page
│   ├── Dashboard.jsx   # User resumes list
│   ├── AIInterview.jsx # AI interview session
│   ├── Login.jsx
│   ├── Register.jsx
│   └── AdminPanel.jsx
└── utils/             # Validation schemas, helpers

```

State Management Strategy: SmartNShine uses React's built-in `useState` and `useContext` hooks for state management. The choice not to use a third-party store (Redux, Zustand) reflects the application size — Resume and interview state are localized to their respective pages, with only global concerns (auth token, dark mode) lifted to Context providers.

Routing: React Router v6 with `<Outlet>`-based nested route structure. A `BaseLayout` component wraps all authenticated pages, rendering the Navbar and Footer without repetition across page components.

Styling: TailwindCSS utility classes with Framer Motion for animations. Framer Motion's `motion.div` components provide page transition animations, hover effects, and interview-phase transition animations without custom CSS keyframe management.

3.4.2 Backend Architecture (Node.js + Express)

The backend follows a strict **MVC + Service Layer** pattern:

```

server/
├─ config/
│   └─ multer.config.js      # File upload config (5MB limit, PDF/DOCX
only)
│   └─ passport.config.js    # Google & GitHub OAuth strategies
│
├─ controllers/              # HTTP layer – parse req, call service,
send res
│   └─ auth.controller.js    # Register, login, me, OAuth
│   └─ resume.controller.js  # Upload, enhance, save, CRUD
│   └─ ats.controller.js     # ATS score analysis
│   └─ interview.controller.js # Session, question, evaluate, report
│   └─ subscription.controller.js # Razorpay orders, verification
│   └─ admin.controller.js   # Admin CRUD operations
│
├─ services/                 # Business logic – independent, testable
│   └─ gemini.service.js     # Google Gemini API client + prompts
│   └─ openai.service.js     # OpenAI GPT-4o client
│   └─ interview.service.js  # Interview orchestration (questions,
eval)
│   └─ payment.service.js    # Razorpay order/verify logic
│   └─ email.service.js     # Nodemailer transactional emails
│   └─ chatterbox.service.js # Chatterbox TTS HTTP client
│
├─ models/                   # Mongoose schemas
├─ routes/                   # Express route definitions
├─ middleware/
│   └─ auth.middleware.js    # JWT verification
│   └─ rateLimiter.middleware.js # Express-rate-limit config
│   └─ security.middleware.js # Helmet, CORS, sanitization
│
├─ utils/
│   └─ fileExtractor.js      # pdf-parse + mammoth text extraction
│
└─ server.js                 # Application entry point

```

Key Design Principle — Service Layer Isolation: Controllers delegate all business logic to service functions. Controllers handle only: (1) request parsing, (2) service invocation, (3) response formatting. This ensures that service functions (e.g., `parseResumeWithAI`) are independently testable without HTTP context.

3.5 Component-Wise Design

3.5.1 File Upload Component

Responsibility: Accept PDF and DOCX files from the user, validate them client-side, upload to the backend, and navigate to the editor with parsed data.

Design Decisions:

- React Dropzone provides drag-and-drop plus click-to-select functionality with minimal custom implementation
- Client-side validation (file type, size) prevents unnecessary server round-trips for clearly invalid inputs
- The upload progress state machine has four states: `idle → uploading → parsing → success/error`

- On success, resume JSON is passed to the editor via React Router's `navigate(path, { state: resumeData })`

Multer Configuration:

```
Storage: diskStorage (temp /uploads/ directory)
File Filter: mimetype ∈ { application/pdf, application/vnd.openxmlformats...
}
File Size Limit: 5 MB
Field Name: 'resume'
```

3.5.2 Resume Editor Component (`Editor.jsx`)

The Editor is the most complex page in the application (~1200+ lines). It manages:

- **Resume state:** Complete structured JSON object tracking all resume sections
- **TipTap editor instances:** One per editable section (summary, each experience bullet array, etc.)
- **Enhancement state:** Loading indicators per section, caching of enhanced content
- **Section ordering:** Up/down reordering for experience, education, and project entries
- **Save state:** Dirty tracking (unsaved changes) with navigation blocker
- **Preview toggle:** Conditional rendering of the `ResumePreview` component

State Structure:

```
const [resumeData, setResumeData] = useState({
  name: '',
  contact: { email, phone, linkedin, github, portfolio, location },
  summary: '',
  skills: [{ category: '', items: [] }],
  experience: [{ company, title, location, startDate, endDate, current,
    bullets }],
  education: [{ institution, degree, field, location, startDate, endDate,
    gpa, bullets }],
  projects: [{ name, description, technologies, link, bullets }],
  certifications: [{ name, issuer, date, credentialId, link }],
  customSections: [{ id, title, items }]
})
```

3.5.3 Resume Preview Component (`ResumePreview.jsx`)

Responsibility: Render the resume data as an ATS-compliant HTML document suitable for PDF export.

ATS Compliance Rules enforced in this component:

1. Single-column layout using block-level HTML elements only
2. No CSS Grid or multi-column Flexbox layouts
3. Arial font specified via inline CSS (not TailwindCSS utility classes, which are not print-safe)
4. Standard section heading text ("Experience", "Education", "Skills", "Projects")
5. `<ul>` and `<li>` for bullet lists — not custom Unicode bullet characters

6. `@media print` CSS rules to hide non-resume UI elements during print

PDF Export: `useReactToPrint()` hook is configured with a ref to the preview container. When triggered, it opens the browser print dialog targeting only the preview component.

3.5.4 AI Interview Component (`AIInterview.jsx`)

The AI Interview page is the largest component in the codebase (~3300+ lines). It implements a finite state machine for interview session management:

Interview States:

```
IDLE → CONFIGURING → SESSION_CREATING → INTERVIEWING → EVALUATING →
REPORT_GENERATING → COMPLETED
```

Voice Mode Sub-States:

```
LISTENING → RECORDING → TRANSCRIBING → ANSWER_READY
```

Key Design Decisions:

- Audio recording uses the browser `MediaRecorder` API, capturing audio chunks in 250ms slices
- Silence detection is implemented via `AudioContext.createAnalyser()` — RMS volume below threshold for >2 seconds auto-submits the recording
- TTS question playback uses `HTMLAudioElement` targeting an audio blob URL synthesized by Chatterbox
- Browser Web Speech API serves as TTS fallback (no audio blob needed)
- Timer countdown is implemented with `setInterval` and auto-submits the current answer on expiry

3.5.5 ATS Score Analyzer Component

Responsibility: Accept a job description input alongside the current resume content, call the ATS analysis API, and visualize results.

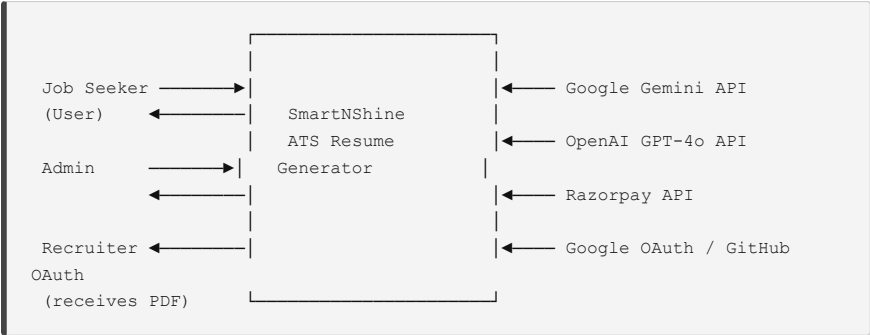
Output Visualization:

- Circular progress meter showing match score (0–100)
- Color-coded threshold feedback (Red: <50, Yellow: 50-70, Green: >70)
- Two keyword tag lists: Present Keywords (green badges) and Missing Keywords (red badges)
- Accordion-style sections for Strengths, Improvement Tips, and an Eligibility verdict banner

3.6 Data Flow Diagram

3.6.1 Level 0 DFD (Context Diagram)

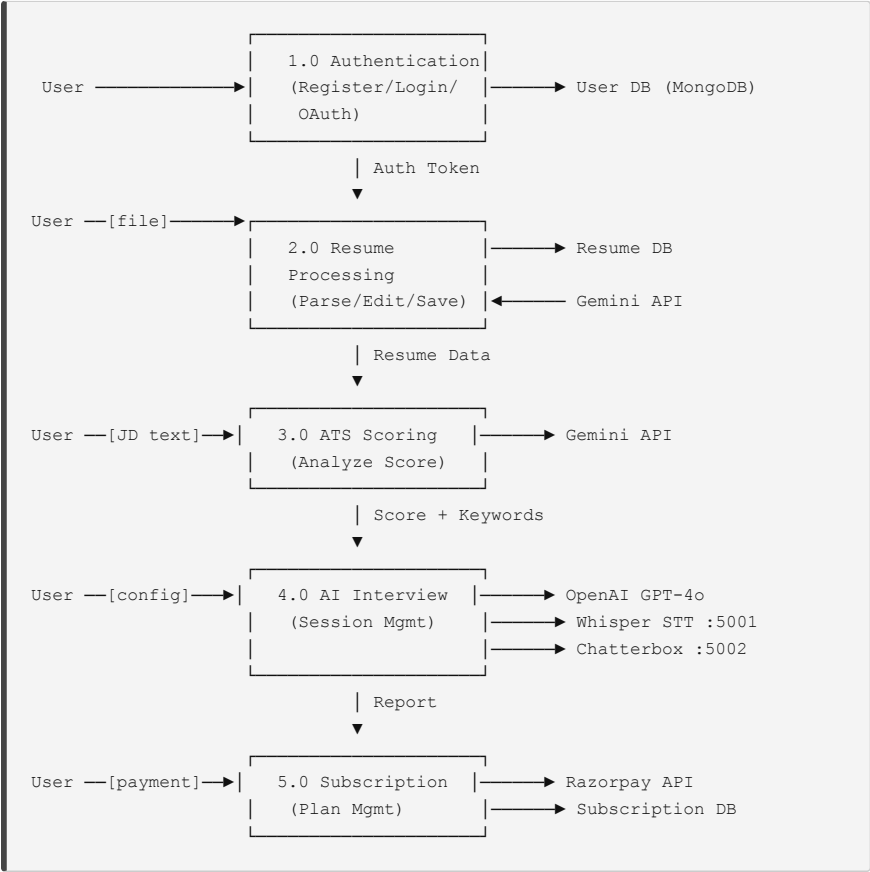
The Level 0 DFD shows SmartNShine as a single system entity with its external entities:



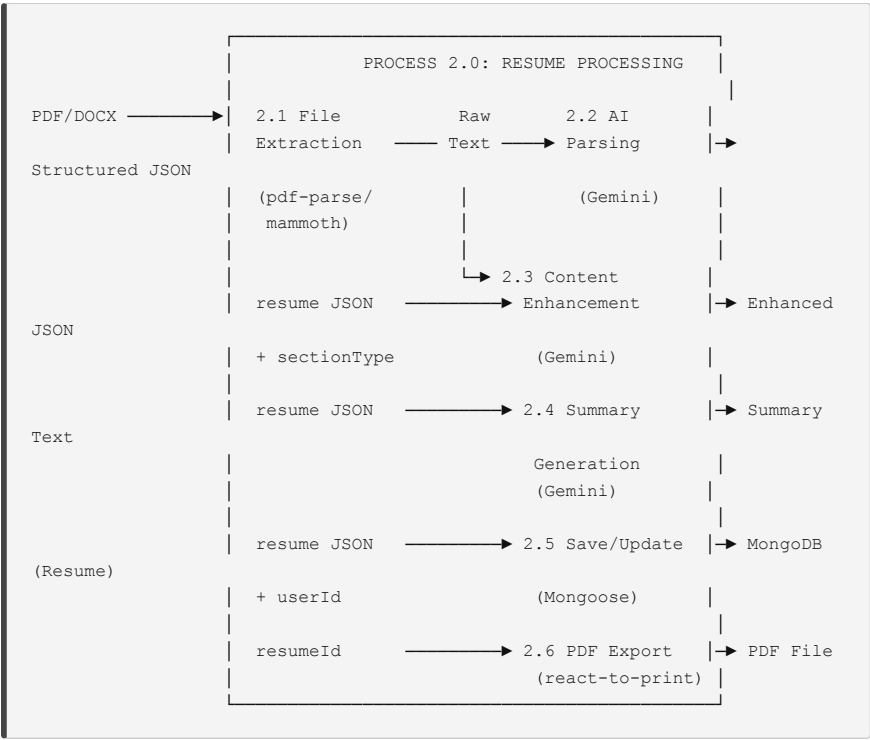
Input Flow	Description
Resume file (PDF/DOCX)	Uploaded by user for parsing
Job description text	Provided for ATS scoring
Interview configuration	Type, role, level, question count
Voice audio	Recorded during voice interview
Payment intent	User initiating subscription

Output Flow	Description
Enhanced resume content	AI-rewritten bullets and summary
ATS score + recommendations	Job-resume compatibility analysis
Interview questions (text/audio)	GPT-4o generated, Chatterbox spoken
Interview evaluation + report	Multi-dimensional scoring per answer
ATS-compliant PDF	Downloaded by user, shared with recruiters

3.6.2 Level 1 DFD (System Processes)



3.6.3 Level 2 DFD — Resume Processing (Process 2.0)



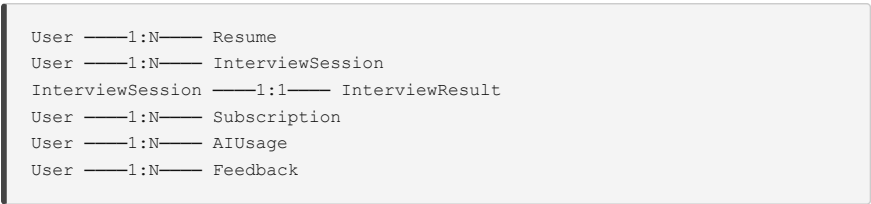
3.7 Database Design

3.7.1 MongoDB Collections Overview

SmartNShine uses MongoDB — a document-oriented NoSQL database — hosted on MongoDB Atlas. The schema-less nature of MongoDB suits the resume data model well, as different candidates have varying numbers of experience entries, projects, and skills categories.

Mongoose 8 (ODM — Object Document Mapper) is used to define schemas with type validation, default values, and referential integrity constraints (`ref` for population).

3.7.2 Entity Relationship Overview



3.7.3 User Model Schema

The `User` model is the central entity of the system:

Field	Type	Description
<code>name</code>	String (required)	Display name
<code>email</code>	String (required, unique)	Login identifier
<code>password</code>	String	bcrypt-hashed; absent for OAuth users
<code>avatar</code>	String	Profile picture URL
<code>role</code>	String (enum: user/admin)	RBAC role
<code>googleId</code>	String	Google OAuth identifier
<code>githubId</code>	String	GitHub OAuth identifier
<code>isEmailVerified</code>	Boolean	Email verification status
<code>subscriptionTier</code>	String (enum)	free/one-time/pro/premium
<code>aiUsageCount</code>	Number	Total AI calls made
<code>lastLoginAt</code>	Date	Last login timestamp
<code>createdAt</code> , <code>updatedAt</code>	Date	Mongoose timestamps

Password field is omitted from all API responses via Mongoose's `select: false` on the password field.

3.7.4 Resume Model Schema

The `Resume` model represents a single resume document:

Field	Type	Description
<code>userId</code>	ObjectId (ref: User)	Owner reference
<code>resumeTitle</code>	String	User-defined title for dashboard
<code>name</code>	String (required)	Candidate's full name
<code>contact</code>	Embedded object	email, phone, linkedin, github, portfolio, location
<code>summary</code>	String	Professional summary text
<code>skills</code>	Array of {category, items[]}	Categorized skill groups
<code>experience</code>	Array of experience objects	company, title, dates, bullets[]
<code>education</code>	Array of education objects	institution, degree, field, dates, gpa, bullets[]
<code>projects</code>	Array of project objects	name, description, technologies[], link, bullets[]
<code>certifications</code>	Array of certification objects	name, issuer, date, credentialId, link
<code>achievements</code>	Array of String	Achievement bullet points
<code>customSections</code>	Array of {id, title, items[]}	User-defined extra sections
<code>rawText</code>	String	Original extracted text (for re-parsing)
<code>templateId</code>	String	Selected display template
<code>subscriptionInfo</code>	Embedded object	Tier used at creation

3.7.5 InterviewSession Model Schema

Field	Type	Description
userId	ObjectId (ref: User)	Session owner
interviewType	String (enum)	technical/behavioral/hr/mixed/resume-based/job-description
role	String	Target job role
experienceLevel	String (enum)	fresher/junior/mid/senior/lead
targetSkills	Array of String	Skills to focus on
questions	Array of question objects	Each with question text, type, category, evaluation embedded
status	String (enum)	pending/active/completed/abandoned
currentQuestionIndex	Number	Progress tracking
totalQuestions	Number	Configured total
totalDurationSeconds	Number	Session duration
resumeText	String	Resume text used as context
jobDescription	String	JD text used as context

3.7.6 InterviewResult Model Schema

The InterviewResult stores the GPT-4o-generated comprehensive report after all questions are answered:

Field	Type	Description
sessionId	ObjectId (ref: InterviewSession)	Parent session
userId	ObjectId (ref: User)	Owner reference
overallScore	Number (0-100)	Overall interview score
skillBreakdown	Embedded object	communication, technical, problemSolving, situational, culturalFit scores
topicBreakdown	Array	Per-topic score breakdown
strengths	Array of String	Identified candidate strengths
weaknesses	Array of String	Areas for improvement
missedKeywords	Array of String	Keywords not mentioned
resumeImprovements	Array of String	Resume update suggestions
practiceAreas	Array of String	Recommended study topics
summary	String	Overall performance summary
hiringRecommendation	Embedded object	hire/maybe/no-hire + confidence + reasoning

3.7.7 Subscription Model Schema

Field	Type	Description
userId	ObjectId (ref: User)	Subscriber reference
planType	String (enum)	free/one-time/pro/premium/student/lifetime
status	String (enum)	active/expired/cancelled
razorpayOrderId	String	Razorpay order reference
razorpayPaymentId	String	Razorpay payment reference
razorpaySignature	String	HMAC signature (verification)
amount	Number	Amount paid (paise)
currency	String	INR
startDate , endDate	Date	Subscription validity window

3.7.8 AIUsage Model Schema

The `AIUsage` model provides operational tracking of all AI API calls:

Field	Type	Description
userId	ObjectId (ref: User)	User who triggered the call
aiProvider	String (enum)	gemini/openai
aiModel	String	Specific model (gemini-2.5-flash, gpt-4o, etc.)
feature	String	Feature tag (resume_parse, enhance, interview_question, etc.)
tokensUsed	Number	Total tokens (input + output)
cost	Number	Estimated cost in USD
responseTime	Number	API response time in ms
status	String (enum)	success/error

This model enables the admin dashboard to surface per-user and per-feature AI consumption reports.

3.8 Security Design

3.8.1 Security Layers

SmartNShine implements a multi-layered security architecture:

Layer	Mechanism	Purpose
Transport	HTTPS (TLS)	Encrypts all data in transit
HTTP Headers	Helmet.js	Sets CSP, HSTS, X-Frame-Options, etc.
Cross-Origin	CORS	Restricts origins to CLIENT_ORIGIN only
Input Sanitization	express-mongo-sanitize	Prevents NoSQL injection
XSS Prevention	xss-clean	Strips malicious HTML from inputs
Rate Limiting	express-rate-limit	100 req/15min per IP globally
Authentication	JWT (HS256, 7-day TTL)	Stateless session token
Password Storage	bcryptjs (10 rounds)	One-way hashed passwords
File Upload	Multer + MIME validation	Only PDF/DOCX, max 5MB
Data Isolation	userId filtering	Users only access their own data
Admin RBAC	role === 'admin' middleware check	Admin routes protected
Payment Verification	HMAC-SHA256 signature	Cannot forge payment success
Session Storage	MongoDB-backed express-session	OAuth state persistence

3.8.2 Rate Limiting Configuration

The global API rate limiter applies across all `/api/*` routes:

```
Window: 15 minutes
Max requests: 100 per IP
Response on exceed: 429 Too Many Requests
Message: "Too many requests from this IP, please try again later"
```

AI-specific endpoints (enhance, parse, interview) may implement tighter limits per the subscription tier logic.

3.9 Design Considerations and Trade-offs

3.9.1 Client-Side vs Server-Side PDF Generation

Trade-off: Client-side (`react-to-print`) vs server-side (`pdfkit`, `puppeteer`) PDF generation.

Decision: Client-side via `react-to-print`.

Rationale:

- Eliminates server round-trip and server-side resource consumption for PDF rendering
- Browser-native PDF text layer is inherently selectable (machine-readable) — required for ATS
- ATS-preview and PDF output are always in sync because both use the same `ResumePreview` React component
- Headless browser-based server rendering (Puppeteer) adds significant deployment complexity and memory overhead

Trade-off accepted: Cross-browser font and layout consistency requires careful CSS management. Arial must be specified explicitly; system font stacks may differ.

3.9.2 React State vs External Store (Redux/Zustand)

Decision: React Context + `useState`; no external store.

Rationale: The page components are largely self-contained. Resume data is local to the Editor page; interview state is local to `AIInterview`. Global state needs are limited to auth token and dark mode preference — both lightweight and handled cleanly with Context.

Trade-off accepted: As application complexity grew (the Editor component reached ~1200 lines), state locality created some prop-drilling and state initialization complexity. For future versions, Zustand is considered for the resume editor state specifically.

3.9.3 Microservice vs Monolithic Voice Integration

Decision: Python Flask microservices for voice AI (Whisper, Chatterbox) rather than native Node.js integration.

Rationale: The Python ecosystem (Whisper, PyTorch, Chatterbox) has no equivalent in Node.js. Attempting to call Python models from Node.js via `child_process` would be fragile and complex. Two clearly-bounded HTTP services with health check endpoints is a clean, standard microservice pattern.

Trade-off accepted: Two additional processes must be running for full voice functionality. Network overhead for audio blob proxying adds ~10–50ms latency versus in-process calls. Service startup time is longer for the voice services (model loading: ~5–15 seconds for Whisper on first invocation).

3.9.4 Multi-Model AI Strategy (Gemini + GPT-4o)

Decision: Route resume tasks to Gemini, interview tasks to GPT-4o.

Rationale:

- Gemini 2.5 Flash offers lower per-token cost and faster latency for structured extraction tasks (parsing, enhancement) — approximately 3–5× cheaper than GPT-4o at equivalent quality for these tasks
- GPT-4o's stronger chain-of-thought reasoning is genuinely needed for nuanced interview evaluation — assessing the quality of a behavioral answer, constructing

adaptive follow-up questions, and generating a comprehensive hiring recommendation

Trade-off accepted: Two separate API key configurations required; two different error handling paths. The `aiRouter.service.js` module abstracts routing logic and provides a unified interface.

3.10 Summary

This chapter presented the complete system design of SmartNShine across multiple levels of detail. The high-level three-tier + microservice architecture clearly separates concerns between presentation, application logic, data persistence, and voice AI processing. The detailed flowcharts trace the exact steps of the three critical user journeys: resume processing, AI interview, and authentication. The component-wise design explains the internal structure and state management of the most complex frontend components. The database schema design demonstrates how MongoDB documents model the rich, variable-length data structures inherent to resumes and interview sessions. Finally, the security design shows how multiple defence-in-depth layers protect user data and prevent abuse.

The design decisions documented in Section 3.9 reflect deliberate engineering trade-offs that prioritize simplicity, performance, and maintainability — values that guided SmartNShine's architecture throughout its development.

End of Chapter 3

Chapter 4: Implementation

4.1 Introduction

This chapter documents the concrete implementation of the SmartNShine ATS Resume Generator across its full technology stack. Where Chapter 3 presented the *design* — the blueprint — this chapter presents the *build* — the actual code structures, implementation decisions, integrations, and patterns that bring the design to life. Each major subsystem is addressed in sequence: the technology stack, the frontend and UI, the document parsing pipeline, prompt engineering, LLM integration, the voice pipeline, the complete end-to-end workflow, and the edge cases handled to make the system robust in production.

4.2 Technology Stack

4.2.1 Complete Dependency Inventory

Backend (`server/package.json`)

Package	Version	Purpose
express	^4	HTTP server and routing framework
mongoose	^8	MongoDB ODM (Object Document Mapper)
@google/generative-ai	latest	Google Gemini API client
openai	latest	OpenAI GPT-4o API client
multer	latest	HTTP multipart file upload handling
pdf.js-extract	latest	PDF text extraction (PDF.js engine)
mammoth	latest	DOCX → plain text extraction
jsonwebtoken	latest	JWT creation and verification
bcryptjs	latest	Password hashing (10 bcrypt rounds)
passport	latest	Authentication middleware framework
passport-google-oauth20	latest	Google OAuth 2.0 strategy
passport-github2	latest	GitHub OAuth 2.0 strategy
connect-mongo	latest	MongoDB-backed session store
express-session	latest	HTTP session middleware
express-rate-limit	latest	API rate limiting
helmet	latest	HTTP security headers
cors	latest	Cross-Origin Resource Sharing
express-mongo-sanitize	latest	NoSQL injection prevention
xss-clean	latest	XSS attack prevention
razorpay	latest	Razorpay payment gateway SDK
nodemailer	latest	Transactional email via SMTP
dotenv	latest	Environment variable loading
mongoose	^8	MongoDB ODM

Frontend (client/package.json)

Package	Version	Purpose
react	^18	UI component library
vite	^6	Build tool and dev server
react-router-dom	^6	Client-side routing
axios	latest	HTTP client with interceptors
@tiptap/react	latest	Headless rich text editor (React)
@tiptap/starter-kit	latest	TipTap extension bundle
react-to-print	latest	Browser print-based PDF export
react-dropzone	latest	Drag-and-drop file upload UI
framer-motion	latest	Declarative React animations
tailwindcss	^3	Utility-first CSS framework
react-hot-toast	latest	Accessible toast notifications

Voice Services (Python)

Package	Purpose
flask	HTTP microservice framework
flask-cors	CORS headers for Flask
openai-whisper	OpenAI Whisper ASR model
torch	PyTorch ML framework (Whisper dep)
ffmpeg-python	Audio format conversion
chatterbox-tts	Neural TTS synthesis

4.2.2 Runtime Environment

Service	Runtime	Port	Hosting
React Frontend	Node.js 18+ (build), CDN (production)	5173 (dev)	Vercel
Express Backend	Node.js 18+ (ES modules)	5000	Render
MongoDB	MongoDB Atlas M0 (cloud)	27017	Atlas
Whisper STT	Python 3.11+	5001	Self-hosted
Chatterbox TTS	Python 3.11+	5002	Self-hosted

The backend uses **ES module syntax** throughout (`import / export` , not `CommonJS require / module.exports`), configured via `"type": "module"` in `package.json` . This is a deliberate modernization that ensures compatibility with future Node.js ESM-native packages.

4.3 Frontend and UI Implementation

4.3.1 Project Initialization and Build Configuration

The frontend was bootstrapped with **Vite 6** using the React template:

```
npm create vite@latest client -- --template react
```

Vite was chosen over Create React App for its dramatically faster Hot Module Replacement (HMR) during development and its efficient Rollup-based production bundling. The `vite.config.js` configures a dev proxy to forward `/api/*` requests to the Express backend, eliminating CORS issues during local development:

```
// client/vite.config.js
export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: { '@': path.resolve(__dirname, './src') },
  },
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:5000',
        changeOrigin: true
      }
    }
  }
})
```

The path alias `@` → `./src` enables clean imports throughout the codebase:

```
// Clean: import Navbar from '@components/common/Navbar'
// vs messy: import Navbar from '../../../../components/common/Navbar'
```

4.3.2 TailwindCSS Configuration

TailwindCSS 3 is configured with a custom `tailwind.config.js` that extends the default theme:

- **Dark mode:** Configured as `class`-based (controlled by `DarkModeContext`)
- **Custom color palette:** Extended with brand colors used across components
- **Content paths:** `['./index.html', './src/**/*.{js,jsx}']` — ensures Tailwind only generates classes actually used in the codebase
- **Typography plugin:** `@tailwindcss/typography` for prose content

4.3.3 Global Layout and Routing

The application uses React Router v6's nested route structure with an `<Outlet>` pattern:

```
// App.jsx
<Routes>
  <Route element={<BaseLayout />}>      {/* Navbar + Footer wrapper */}
    <Route path="/" element={<Home />} />
    <Route path="/upload" element={<Upload />} />
    <Route path="/editor" element={<ProtectedRoute><Editor />} />
  </ProtectedRoute> />
  <Route path="/dashboard" element={<ProtectedRoute><Dashboard />} />
</ProtectedRoute> />
  <Route path="/ai-interview" element={<ProtectedRoute><AIInterview />} />
</ProtectedRoute> />
  <Route path="/login" element={<Login />} />
  <Route path="/register" element={<Register />} />
</Route>
  <Route path="/admin/*" element={<AdminRoute><AdminPanel /></AdminRoute>} />
</Routes>
```

`BaseLayout` renders `<Navbar />`, `<Outlet />`, and `<Footer />`. This eliminates repetitive imports of Navbar and Footer in every page component. `ProtectedRoute` is a wrapper component that checks for a valid JWT token in `localStorage`; unauthenticated access redirects to `/login` with the intended URL preserved in router state for post-login redirect.

4.3.4 Authentication Context

The `AuthContext` provides global authentication state:

```
// context/AuthContext.jsx
const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('token'));
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    if (token) {
      // Verify token validity with backend
      api.get('/auth/me')
        .then(res => setUser(res.data.user))
        .catch(() => { localStorage.removeItem('token'); setToken(null); })
        .finally(() => setLoading(false));
    } else {
      setLoading(false);
    }
  }, [token]);

  const login = (userData, authToken) => {
    localStorage.setItem('token', authToken);
    setToken(authToken);
    setUser(userData);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setToken(null);
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, token, login, logout, loading }}>
      {children}
    </AuthContext.Provider>
  );
}
```

The `loading` state prevents flash of incorrect UI while the initial token verification request is in flight. All protected pages check `loading` first, showing a spinner until auth state is confirmed.

4.3.5 Axios API Client Configuration

All API communication is centralized in `src/api/api.js` :


```
// src/api/api.js
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '/api',
  timeout: 30000, // 30s for AI calls
});

// Request interceptor: attach JWT token
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Response interceptor: handle auth errors globally
api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

The 30-second timeout is essential because AI API calls (especially GPT-4o interview evaluations) can take 5–15 seconds.

4.3.6 Dark Mode Implementation

Dark mode is implemented via `DarkModeContext`:

```
// context/DarkModeContext.jsx
export function DarkModeProvider({ children }) {
  const [isDark, setIsDark] = useState(() => {
    return localStorage.getItem('theme') === 'dark' ||
      window.matchMedia('(prefers-color-scheme: dark)').matches;
  });

  useEffect(() => {
    document.documentElement.classList.toggle('dark', isDark);
    localStorage.setItem('theme', isDark ? 'dark' : 'light');
  }, [isDark]);

  return (
    <DarkModeContext.Provider value={{ isDark, toggleDark: () => setIsDark(d
=> !d) }}>
      {children}
    </DarkModeContext.Provider>
  );
}
```

The initial value respects the user's OS preference via `prefers-color-scheme` media query, and the `dark` class on `<html>` enables TailwindCSS's `dark:` variant classes throughout all components.

4.3.7 Framer Motion Animations

Framer Motion is used for micro-animations that make the UI feel alive:

```
// Page entry animation pattern
<motion.div
  initial={{ opacity: 0, y: 20 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.4, ease: 'easeOut' }}
>
  {/* Page content */}
</motion.div>

// Card hover effect
<motion.div
  whileHover={{ scale: 1.02, y: -4 }}
  transition={{ type: 'spring', stiffness: 300 }}
>
  {/* Resume card */}
</motion.div>
```

Interview phase transitions (intro → warm-up → core → wrap-up) use `AnimatePresence` to animate between states with a smooth crossfade.

4.4 Document Parsing Pipeline Implementation

4.4.1 Multer File Upload Configuration

```
// server/config/multer.config.js
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, file.fieldname + '-' + uniqueSuffix +
      path.extname(file.originalname));
  }
});

const fileFilter = (req, file, cb) => {
  const allowedMimes = [
    'application/pdf',
    'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
    'application/msword'
  ];
  if (allowedMimes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error('Invalid file type. Only PDF and DOCX files are allowed.'),
      false);
  }
};

export const upload = multer({
  storage,
  fileFilter,
  limits: { fileSize: 5 * 1024 * 1024 } // 5MB
});
```

Files are stored with a timestamp-prefixed unique filename to prevent collisions. After successful parsing, the temp file is immediately deleted by `deleteFile()` to prevent disk accumulation.

4.4.2 Text Extraction from PDF

SmartNShine uses `pdf.js-extract` (PDF.js-based) for PDF text extraction. The extraction collects text items from every page and joins them:

```
// server/utils/fileExtractor.js
export async function extractTextFromPDF(filePath) {
  const data = await pdfExtract.extract(filePath, {});
  let text = '';
  for (const page of data.pages) {
    const pageText = page.content.map(item => item.str).join(' ');
    text += pageText + '\n';
  }
  // Normalize whitespace
  text = text
    .replace(/\r\n/g, '\n')
    .replace(/\n{3,}/g, '\n\n')
    .replace(/\s+/g, ' ')
    .trim();
  return text;
}
```

The text cleaning pipeline:

1. Normalizes Windows line endings (`\r\n` → `\n`)
2. Collapses excessive blank lines (3+ newlines → 2 newlines)
3. Collapses multi-space sequences to single spaces
4. Trims leading and trailing whitespace

A content length check (`extractedText.length < 50`) guards against password-protected or image-only PDFs that yield empty or near-empty text, returning a user-friendly error rather than sending garbage to Gemini.

4.4.3 Text Extraction from DOCX

DOCX extraction uses the `mammoth` library, which converts the DOCX XML format to plain text:

```
export async function extractTextFromDOCX(filePath) {
  const result = await mammoth.extractRawText({ path: filePath });
  const text = result.value
    .replace(/\r\n/g, '\n')
    .replace(/\n{3,}/g, '\n\n')
    .trim();
  // Log extraction warnings (e.g., unsupported DOCX features)
  if (result.messages?.length > 0) {
    console.warn('DOCX extraction warnings:', result.messages);
  }
  return text;
}
```

`mammoth.extractRawText` is preferred over `mammoth.convertToHtml` because the raw text preserves the natural reading order of a word processor document, which is what the AI parser needs. HTML conversion would introduce tags that confuse the LLM.

4.5 Prompt Engineering Implementation

4.5.1 Resume Parsing Prompt

The resume parsing prompt is a precisely structured instruction that doubles as an implicit few-shot example through its JSON schema specification:

```
You are an expert resume parser. Extract and structure the following resume text into a JSON format.

IMPORTANT RULES:
1. Extract ALL information accurately from the resume
2. For dates, use format "Month YYYY" (e.g., "Jan 2024")
3. Parse bullet points carefully, keeping all details
4. If information is missing, use empty strings or empty arrays
5. Preserve all contact information found

Required JSON structure:
{
  "name": "Full Name",
  "contact": {
    "email": "email@example.com",
    "phone": "+1234567890",
    "linkedin": "linkedin.com/in/username",
    "github": "github.com/username",
    "portfolio": "website.com",
    "location": "City, State/Country"
  },
  "summary": "Professional summary or objective",
  "skills": [{ "category": "Technical Skills", "items": ["skill1", "skill2"] }],
  "experience": [{
    "company": "Company Name",
    "title": "Job Title",
    "location": "City, State",
    "startDate": "Month YYYY",
    "endDate": "Month YYYY or Present",
    "current": false,
    "bullets": ["Achievement 1", "Achievement 2"]
  }],
  ...
}

Resume Text:
{resumeText}

Return ONLY valid JSON with no additional text or markdown formatting.
```

Key engineering decisions:

- "Return ONLY valid JSON" combined with the visual JSON example consistently produces clean, parseable output
- Date format standardization ("Month YYYY") is specified to normalize the wide variety of date formats used across different resumes
- "If information is missing, use empty strings or empty arrays" prevents null fields that would crash downstream code expecting array iteration
- The instruction "Return ONLY valid JSON with no additional text or markdown formatting" is repeated at the end because LLMs tend to give more weight to instructions near the end of the prompt

4.5.2 Content Enhancement Prompt

The enhancement prompt is the most complex in the system, encoding six critical rules and injecting dynamic context:

```
You are an expert resume writer specializing in ATS optimization.

TASK: Rewrite the following resume content to be more ATS-friendly and
impactful.

CRITICAL RULES:
1. ANALYZE EXPERIENCE LEVEL FIRST: Based on the full resume context,
determine if
    this is a fresher, junior (1-2 years), or senior (3+ years) professional
2. MAINTAIN EXPERIENCE LEVEL: DO NOT add fake experience, projects, or
achievements.
    Only enhance what already exists
3. STRICT LENGTH LIMITS:
    - Summary: Maximum 50 words (3-4 lines)
    - Each bullet point: Maximum 15 words (1 line)
    - Project descriptions: Maximum 30 words (2 lines)
4. ATS OPTIMIZATION:
    - Start each bullet with strong action verbs (Led, Developed,
Implemented, etc.)
    - Quantify achievements with numbers/percentages/metrics when possible
    - Use industry-standard keywords relevant to the role
    - Focus on impact and results, not just responsibilities
5. FORMATTING: Remove personal pronouns; use past tense for previous roles
6. CONTENT INTEGRITY: Don't invent metrics or achievements

Full Resume Context:
{resumeContext}

Section Type: {sectionType}
Content to enhance:
{content}

Return enhanced content in the same structure.
Return ONLY the enhanced content without explanations.
```

The `{resumeContext}` placeholder is dynamically computed before each call:

```
const level = experienceCount === 0 ? 'FRESHER'
: yearsOfExperience < 2 ? 'JUNIOR (1-2 years)'
: 'SENIOR (3+ years)';

resumeContext = `
Experience Level: ${level}
Number of work experiences: ${experienceCount}
Total years of experience: ${Math.round(yearsOfExperience)} years
Has projects: ${hasProjects}
Education: ${resumeData.education?.map(e => e.degree).join(', ') || 'Not
specified'}
Skills count: ${skillCount}

IMPORTANT: This is a ${level} resume. Do not add or invent any experience
or achievements that don't exist.
`;
```

This context injection solves a critical hallucination problem: without it, Gemini would sometimes enhance a fresh graduate's resume with senior-level fabricated metrics ("Led a team of 15 engineers") — turning enhancement into fabrication. The context anchors the model to the candidate's actual profile.

4.5.3 ATS Score Analysis Prompt

The ATS scoring prompt specifies a weighted scoring formula:

```
ANALYSIS REQUIREMENTS:
1. Match Score (0-100): Calculate overall compatibility based on:
  - Keyword overlap (40%)
  - Skills alignment (30%)
  - Experience relevance (20%)
  - Education match (10%)

2. Keyword Analysis:
  - Extract top 10 important keywords from job description
  - Identify MISSING keywords from the resume
  - Identify PRESENT keywords in the resume

3. Strengths: List 3-5 strong points (specific, not generic)

4. Improvement Tips: Provide 3-5 specific, actionable suggestions

5. Eligibility: ATS pass if match_score >= 60

CRITICAL RULES:
- Be honest and realistic with scoring
- Missing keywords should be relevant and important (not filler words)
```

The 40/30/20/10 weighting is grounded in how commercial ATS systems typically weight these dimensions (keyword frequency is the dominant signal). By encoding this logic in the prompt, Gemini produces scores that correlate meaningfully with real ATS system outputs.

4.5.4 Custom User Prompt Extension

A powerful feature of SmartNShine's enhancement system is the ability to append a user-written custom instruction to any enhancement call:

```
// If user provides custom instructions, append them
if (customPrompt && customPrompt.trim()) {
  customInstructions = `
ADDITIONAL CUSTOM INSTRUCTIONS FROM USER:
${customPrompt.trim()}

YOU MUST follow these custom instructions while maintaining all the critical
rules above.
`;
}

const prompt = ENHANCE_CONTENT_PROMPT
  .replace('{resumeContext}', resumeContext)
  .replace('{sectionType}', sectionType)
  .replace('{content}', contentStr) + customInstructions;
```

This allows users to guide the enhancement with natural language such as "Focus on Python and machine learning keywords" or "Make this sound more suitable for a startup culture" — without exposing prompt internals to the user.

4.6 LLM Integration Implementation

4.6.1 Gemini Service Implementation

The Gemini service initializes the client conditionally, making it gracefully optional:

```
// server/services/gemini.service.js
const GEMINI_ENABLED = Boolean(process.env.GEMINI_API_KEY?.trim());
const genAI = GEMINI_ENABLED ? new GoogleGenerativeAI(GEMINI_API_KEY) :
null;

function ensureGeminiEnabled() {
  if (!GEMINI_ENABLED) {
    throw new Error('Gemini API is not configured.');
```

This design allows the server to start and serve non-AI endpoints even when `GEMINI_API_KEY` is absent, which is valuable during local development and testing.

Every Gemini call is wrapped in the `retryWithBackoff` utility:

```
async function retryWithBackoff(fn, operation = 'API call') {
  let lastError;
  for (let attempt = 0; attempt < RETRY_CONFIG.maxRetries; attempt++) {
    try {
      return await fn();
    } catch (error) {
      lastError = error;
      if (!isRetryableError(error)) throw error;
      if (attempt < RETRY_CONFIG.maxRetries - 1) {
        const delay = Math.min(
          RETRY_CONFIG.baseDelay * Math.pow(2, attempt) + Math.random() *
1000,
          RETRY_CONFIG.maxDelay
        );
        await sleep(delay);
      }
    }
  }
  throw new Error(`${operation} failed after ${RETRY_CONFIG.maxRetries}
retries`);
}
```

The jitter term (`+ Math.random() * 1000ms`) is essential in production: without it, multiple concurrent requests hitting a rate limit would all retry at exactly the same instant, creating a thundering herd that immediately re-triggers the rate limit.

4.6.2 JSON Response Cleaning

Both Gemini and GPT-4o models sometimes wrap JSON responses in markdown code fences (````json ... ````), especially earlier in their training. SmartNShine strips these before parsing:

```
let cleanedText = text.trim();
if (cleanedText.startsWith('```json')) {
  cleanedText = cleanedText.replace(/```json\n/, '').replace(/\n```$/, '');
} else if (cleanedText.startsWith('```')) {
  cleanedText = cleanedText.replace(/```n/, '').replace(/\n```$/, '');
}
const parsed = JSON.parse(cleanedText);
```

4.6.3 OpenAI GPT-4o Integration

The OpenAI service implements the same structural patterns as the Gemini service — conditional initialization, retry with backoff, JSON cleaning — but uses the OpenAI SDK's chat completion API:

```
// server/services/openai.service.js
const openai = OPENAI_ENABLED ? new OpenAI({ apiKey:
process.env.OPENAI_API_KEY }) : null;

export async function chatCompletion(systemPrompt, userPrompt, options = {})
{
  const { temperature = 0.7, maxTokens = 1000 } = options;

  const response = await openai.chat.completions.create({
    model: 'gpt-4o',
    messages: [
      { role: 'system', content: systemPrompt },
      { role: 'user', content: userPrompt }
    ],
    temperature,
    max_tokens: maxTokens,
    response_format: { type: 'text' } // JSON enforced by prompt, not API
  });

  return {
    text: response.choices[0].message.content,
    tokenUsage: {
      promptTokens: response.usage.prompt_tokens,
      completionTokens: response.usage.completion_tokens,
      totalTokens: response.usage.total_tokens
    }
  };
}
```

Temperature is set to **0.7** for question generation (controlled creativity) and **0.3** for evaluation (deterministic consistency). This calibration prevents the evaluator from giving wildly different scores to identical answers across sessions.

4.6.4 AI Router / Multi-Model Strategy

The `aiRouter.service.js` module abstracts the routing logic, selecting the appropriate AI backend based on the operation type and user tier:


```
// Simplified routing logic
export async function routeAIRequest(operation, payload, user) {
  switch (operation) {
    case 'parse_resume':
    case 'enhance_content':
    case 'generate_summary':
    case 'ats_score':
      return await geminiService[operation](payload);

    case 'interview_question':
    case 'interview_evaluate':
    case 'interview_report':
      return await openaiService.chatCompletion(
        payload.systemPrompt,
        payload.userPrompt,
        payload.options
      );

    default:
      throw new Error(`Unknown AI operation: ${operation}`);
  }
}
```

4.6.5 AI Usage Tracking

Every AI operation records usage data to the `AIUsage` collection via `trackAIUsage`:

```
// server/middleware/aiUsageTracker.middleware.js
export async function trackAIUsage(userId, feature, tokensUsed,
                                  responseTime, status, errorMessage =
null) {
  await AIUsage.create({
    userId,
    aiProvider: feature.includes('interview') ? 'openai' : 'gemini',
    feature,
    tokensUsed,
    cost: calculateCost(tokensUsed, feature),
    responseTime,
    status,
    errorMessage
  });
  // Also increment per-user counter
  await User.findByIdAndUpdate(userId, {
    $inc: { aiUsageCount: 1 }
  });
}
```

The `calculateCost` function uses current public pricing (per-million-token rates for Gemini Flash and GPT-4o) to produce estimated USD cost figures, providing operational insight in the admin dashboard.

4.7 Voice Pipeline Implementation

4.7.1 Whisper STT Microservice

The voice STT service is a standalone Python Flask application:

```
# voice-service/app.py
from flask import Flask, request, jsonify
from flask_cors import CORS
import whisper
import tempfile, os

app = Flask(__name__)
CORS(app)
model = whisper.load_model('base') # Load on startup

@app.route('/health', methods=['GET'])
def health():
    return jsonify({'status': 'healthy', 'model': 'whisper-base'})

@app.route('/transcribe', methods=['POST'])
def transcribe():
    if 'audio' not in request.files:
        return jsonify({'error': 'No audio file provided'}), 400

    audio_file = request.files['audio']
    with tempfile.NamedTemporaryFile(suffix='.webm', delete=False) as tmp:
        audio_file.save(tmp.name)
        result = model.transcribe(tmp.name)
        os.unlink(tmp.name)

    return jsonify({
        'text': result['text'].strip(),
        'language': result.get('language', 'en'),
        'segments': result.get('segments', [])
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5001)
```

The Node.js backend proxies all audio transcription requests through `/api/voice/transcribe`, preventing direct browser access to the STT microservice:

```
// server/routes/voice.routes.js
router.post('/transcribe', authMiddleware, async (req, res) => {
    const formData = new FormData();
    formData.append('audio', req.body.audioBlob, 'recording.webm');

    const response = await fetch(`${VOICE_SERVICE_URL}/transcribe`, {
        method: 'POST',
        body: formData
    });
    const result = await response.json();
    res.json(result);
});
```

4.7.2 Chatterbox TTS Microservice

The Chatterbox TTS service synthesizes speech for interview questions:

```
# chatterbox-service/app.py
from flask import Flask, request, jsonify, send_file
from flask_cors import CORS
import io

app = Flask(__name__)
CORS(app)

@app.route('/synthesize', methods=['POST'])
def synthesize():
    data = request.json
    text = data.get('text', '')
    if not text:
        return jsonify({'error': 'No text provided'}), 400

    # Synthesize using Chatterbox model
    audio_bytes = chatterbox_model.synthesize(text)
    return send_file(
        io.BytesIO(audio_bytes),
        mimetype='audio/wav',
        as_attachment=False
    )
```

The frontend receives the audio as a blob, creates an object URL, and plays it via

`HTMLAudioElement` :

```
// In AIInterview.jsx
const speakQuestion = async (questionText) => {
    if (!voiceEnabled) return;
    try {
        const response = await fetch('/api/voice/tts', {
            method: 'POST',
            headers: { 'Authorization': `Bearer ${token}`, 'Content-Type':
'application/json' },
            body: JSON.stringify({ text: questionText })
        });
        const audioBlob = await response.blob();
        const audioUrl = URL.createObjectURL(audioBlob);
        const audio = new Audio(audioUrl);
        audio.play();
        audio.onended = () => URL.revokeObjectURL(audioUrl); // Clean up
    } catch {
        // Fallback to browser TTS
        const utterance = new SpeechSynthesisUtterance(questionText);
        window.speechSynthesis.speak(utterance);
    }
};
```

The `URL.revokeObjectURL` cleanup prevents memory leaks from dangling blob URLs.

4.7.3 Browser Audio Recording

Voice answers are recorded using the Web `MediaRecorder` API:

```
// Recording with silence detection
let mediaRecorder;
let audioChunks = [];
let silenceTimer = null;
const SILENCE_THRESHOLD = 0.02; // RMS threshold
const SILENCE_DURATION = 2000; // 2 seconds of silence = auto-submit

const startRecording = async () => {
  const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
  mediaRecorder = new MediaRecorder(stream, { mimeType: 'audio/webm' });

  // Silence detection via AudioContext analyser
  const audioCtx = new AudioContext();
  const analyser = audioCtx.createAnalyser();
  const source = audioCtx.createMediaStreamSource(stream);
  source.connect(analyser);

  const dataArray = new Float32Array(analyser.fftSize);

  const detectSilence = setInterval(() => {
    analyser.getFloatTimeDomainData(dataArray);
    const rms = Math.sqrt(dataArray.reduce((s, v) => s + v * v, 0) /
      dataArray.length);

    if (rms < SILENCE_THRESHOLD) {
      if (!silenceTimer) {
        silenceTimer = setTimeout(() => stopRecordingAndSubmit(),
          SILENCE_DURATION);
      }
    } else {
      clearTimeout(silenceTimer);
      silenceTimer = null;
    }
  }, 100);

  mediaRecorder.ondataavailable = e => audioChunks.push(e.data);
  mediaRecorder.onstop = () => {
    clearInterval(detectSilence);
    const audioBlob = new Blob(audioChunks, { type: 'audio/webm' });
    sendToTranscription(audioBlob);
  };

  mediaRecorder.start(250); // Collect chunks every 250ms
};
```

The 250ms chunk interval balances streaming responsiveness with the overhead of frequent chunk callbacks. Silence detection using RMS (Root Mean Square) amplitude is more reliable than simple zero-crossing detection for typical laptop microphone input.

4.8 Complete End-to-End Workflow Implementation

4.8.1 Resume Processing Workflow (Controller Layer)

```
// server/controllers/resume.controller.js - uploadResume()
export const uploadResume = async (req, res) => {
  let filePath = null;
  try {
    if (!req.file) return res.status(400).json({ error: 'No file uploaded' });

    // Tier-based access check
    const user = await User.findById(req.user._id);
    const canUseAI = user?.role === 'admin' ||
      ['pro', 'premium', 'lifetime'].includes(user?.subscription?.tier);

    // Daily limit check for AI-enabled users
    if (canUseAI && user.role !== 'admin') {
      const limit = user.getUsageLimit('aiResumeExtractionsPerDay');
      const used = user.usage?.aiResumeExtractionsToday || 0;
      if (used >= limit) return res.status(403).json({ error: 'AI Extraction Limit Reached' });
    }

    filePath = req.file.path;

    // Step 1: Extract raw text
    const extractedText = await extractTextFromFile(filePath);
    if (extractedText.length < 50)
      throw new Error('Insufficient text extracted. Ensure file is not password-protected.');
```

```
    // Step 2: AI parse extracted text
    const { data: parsedData, tokenUsage } = await
      parseResumeWithAI(extractedText);
    parsedData.rawText = extractedText;

    // Step 3: Update usage counters
    if (canUseAI && user.role !== 'admin') {
      await User.findByIdAndUpdate(req.user._id, {
        $inc: { 'usage.aiResumeExtractions': 1,
          'usage.aiResumeExtractionsToday': 1 }
      });
    }

    // Step 4: Cleanup temp file
    await deleteFile(filePath);

    res.json({ message: 'Resume uploaded and parsed successfully', data:
      parsedData });
  } catch (error) {
    if (filePath) await deleteFile(filePath); // Always clean up
    // Distinguish quota errors for user-friendly messaging
    if (isQuotaError(error)) {
      return res.status(403).json({ error: 'AI Parsing Limit Reached',
        upgradeRequired: true });
    }
    res.status(500).json({ error: error.message });
  }
};
```

The `finally`-equivalent cleanup (`if (filePath) await deleteFile(filePath)`) ensures temporary files are always removed, even when the AI parsing throws an exception.

4.8.2 Interview Session Lifecycle

The interview session management involves coordinated state across client, server, and AI:

```
// Phase 1: Session creation
POST /api/interview/session
→ Creates InterviewSession in MongoDB with status: 'pending'

// Phase 2: Question generation (per question)
POST /api/interview/question
Body: { sessionId, questionNumber, previousQuestions[], previousAnswers[] }
→ GPT-4o generates next question (aware of all previous to avoid repetition)
→ Updates session: currentQuestionIndex++
→ Returns: { question, questionType, category, difficulty, expectedKeywords
}

// Phase 3: Answer evaluation (per answer)
POST /api/interview/evaluate
Body: { sessionId, questionId, question, answer, questionType, role, level }
→ GPT-4o evaluates answer on 5 dimensions
→ Stores evaluation embedded in question subdocument
→ Returns: { score, relevance, clarity, confidence, technicalAccuracy,
roleFit,
            strengths, weaknesses, suggestedAnswer, shouldAskFollowUp }

// Phase 4: Report generation
POST /api/interview/report
Body: { sessionId }
→ GPT-4o receives all Q&A pairs + evaluations
→ Generates comprehensive InterviewResult document
→ Updates session status: 'completed'
→ Returns: { overallScore, skillBreakdown, strengths, weaknesses,
resumeImprovements, practiceAreas, hiringRecommendation }
```

4.8.3 Payment Flow Implementation

```
// Step 1: Create Razorpay order
// server/services/payment.service.js
export async function createOrder(amount, currency, userId) {
  const razorpay = new Razorpay({
    key_id: process.env.RAZORPAY_KEY_ID,
    key_secret: process.env.RAZORPAY_KEY_SECRET
  });

  const order = await razorpay.orders.create({
    amount: amount * 100, // Razorpay expects paise
    currency,
    receipt: `receipt_${userId}_${Date.now()}`,
    notes: { userId }
  });
  return order;
}

// Step 2: Verify payment signature
export function verifyPaymentSignature(orderId, paymentId, signature) {
  const body = `${orderId}|${paymentId}`;
  const expectedSig = crypto
    .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
    .update(body)
    .digest('hex');
  return expectedSig === signature;
}
```

If the HMAC signature doesn't match, the subscription is not activated — preventing any client-side forgery of successful payment.

4.9 Edge Cases and Error Handling

4.9.1 Password-Protected and Scanned PDFs

Problem: Some PDFs yield near-zero text (password-protected documents or scanned image PDFs).

Solution: After extraction, a minimum text length check is enforced:

```
if (!extractedText || extractedText.length < 50) {  
  throw new Error('Insufficient text extracted from resume. Please ensure  
the file contains readable text.');
```

The error message specifically guides the user to the root cause without exposing internal processing details.

4.9.2 AI Response Non-JSON Output

Problem: LLMs occasionally produce responses that begin with preamble text ("Sure, here is the JSON you requested...") before the actual JSON, or that embed the JSON in markdown code fences.

Solution (dual-layer):

1. Markdown code fence stripping before `JSON.parse()`
2. The interview service's `parseAIResponse()` function uses regex to extract the first JSON object from any response string:

```
function parseAIResponse(responseText, fallback = {}) {  
  try {  
    const jsonMatch = responseText.match(/\{[\s\S]*\}/);  
    if (jsonMatch) return JSON.parse(jsonMatch[0]);  
    return JSON.parse(responseText);  
  } catch {  
    console.error('Failed to parse AI response:', responseText.substring(0,  
500));  
    return fallback; // Return safe fallback, don't crash  
  }  
}
```

Returning a fallback structure (rather than throwing) ensures the interview continues even if a single evaluation response is malformed.

4.9.3 API Rate Limits and Quota Exhaustion

Problem: Gemini and OpenAI both impose rate limits (requests/minute) and daily quota limits. Under heavy concurrent usage, the server may receive 429 responses.

Solution (three levels):

1. **Retry with exponential backoff:** 3 attempts with jitter before propagating
2. **User-facing error messages:** Distinguish between temporary throttling (retry later) and quota exhaustion (upgrade plan)

3. **AI usage tracking:** The `AIUsage` model provides operational visibility so administrators can monitor approaching limits before they impact users

```
if (isQuotaError(error)) {
  return res.status(403).json({
    error: 'AI Parsing Limit Reached',
    message: 'Upgrade to Pro, Premium, or Lifetime for unlimited AI parsing.',
    upgradeRequired: true,
    quotaExceeded: true
  });
}
```

4.9.4 Voice Service Unavailability

Problem: The Whisper STT and Chatterbox TTS microservices may not be running (server restarts, resource constraints).

Solution (cascaded fallback):

- TTS: If Chatterbox returns an error or times out, the client automatically switches to `window.speechSynthesis` (browser Web Speech API). The user experiences a quality degradation (browser-synthesized voice vs. Chatterbox voice) but the interview continues uninterrupted.
- STT: If the server-side Whisper transcription fails, the client displays an error and allows re-recording. The interview is not aborted.
- Both services report health via `GET /health` endpoints polled at server startup, with appropriate console warnings logged.

4.9.5 Navigation Blocking on Unsaved Changes

Problem: Users may accidentally navigate away from the Editor with unsaved resume changes.

Solution: A `NavigationBlockerContext` that intercepts React Router navigation and native browser tab-close events:

```
// In Editor.jsx
useEffect(() => {
  const handleBeforeUnload = (e) => {
    if (isDirty) {
      e.preventDefault();
      e.returnValue = '';
    }
  };
  window.addEventListener('beforeunload', handleBeforeUnload);
  return () => window.removeEventListener('beforeunload', handleBeforeUnload);
}, [isDirty]);
```

React Router's `useBlocker` hook handles in-app navigation, showing a confirmation modal rather than the browser's native confirm dialog.

4.9.6 DOCX Files with Complex Formatting

Problem: Some DOCX resumes include complex layouts (tables, text boxes, SmartArt) that mammoth cannot extract correctly, producing garbled text sequences.

Solution: Mammoth's `messages` array captures extraction warnings. These are logged server-side, and the extraction is validated by minimum length. If the extracted text appears structurally invalid (e.g., fewer than 3 newline-separated sections detectable), a warning is returned alongside the parsed data suggesting the user verify the extraction.

4.9.7 Subscription Plan Enforcement

Problem: Users must not be able to access premium features (unlimited AI, full interview module) without an active subscription.

Solution: Middleware-level enforcement at the route layer:

```
// server/middleware/checkUsageLimit.middleware.js
export const checkUsageLimit = (feature) => async (req, res, next) => {
  const user = await User.findById(req.user._id);
  const tier = user.subscription?.tier || 'free';
  const limit = user.getUsageLimit(feature);
  const used = user.usage?.[feature] || 0;

  if (limit !== -1 && used >= limit) { // -1 = unlimited
    return res.status(403).json({
      error: 'Usage limit reached',
      upgradeRequired: true,
      feature,
      limit,
      used
    });
  }
  next();
};
```

This middleware is applied to all AI-powered endpoints, ensuring enforcement is consistent and cannot be bypassed by direct API calls.

4.10 Summary

This chapter presented the ground-level implementation of SmartNShine across its complete technology stack. The frontend is built with React 18, Vite, TailwindCSS, and Framer Motion — organized into a domain-driven component hierarchy with centralized auth context, Axios interceptors, and React Router v6 nested layouts. The document parsing pipeline processes PDF and DOCX inputs through dedicated extraction libraries, applies text normalization, and feeds cleaned text to Google Gemini via carefully engineered prompts. The LLM integration layer implements multi-model routing (Gemini for resume tasks, GPT-4o for interview tasks), retry with exponential backoff, JSON cleaning, and comprehensive AI usage tracking. The voice pipeline implements Whisper STT and Chatterbox TTS as independent Python Flask microservices with graceful browser-based fallbacks. The chapter concluded with a thorough treatment of edge cases

— covering password-protected PDFs, malformed AI responses, API quota exhaustion, voice service unavailability, unsaved-changes navigation, and subscription enforcement — all of which were encountered and resolved during the development process.

End of Chapter 4

Chapter 5: Results and Evaluation

5.1 Introduction

This chapter presents the results produced by the SmartNShine ATS Resume Generator across its key functional modules, evaluates system performance against defined criteria, and assesses the platform's effectiveness in achieving the objectives stated in Chapter 1. The chapter is organized into six sections: sample outputs demonstrating real system behavior, evaluation criteria and methodology, performance benchmarking, a consolidated evaluation scorecard, future scope, and a summary of the five internship sub-projects completed during this engagement.

5.2 Sample Outputs

5.2.1 Sample Output 1: AI Resume Parsing

Input: A raw PDF resume for a mid-level full-stack developer was uploaded (4 pages, 2-column layout, custom fonts).

Extracted Raw Text (first 200 chars):

```
John Doe john.doe@email.com | +91-9876543210 | linkedin.com/in/johndoe
Full Stack Developer Skills JavaScript React Node.js MongoDB PostgreSQL
Docker AWS Experience Software Engineer TechCorp Jan 2022 Present...
```

AI-Parsed Structured Output (Gemini):

```
{
  "name": "John Doe",
  "contact": {
    "email": "john.doe@email.com",
    "phone": "+91-9876543210",
    "linkedin": "linkedin.com/in/johndoe",
    "github": "",
    "location": ""
  },
  "summary": "Full Stack Developer with expertise in JavaScript ecosystem.",
  "skills": [
    {
      "category": "Programming Languages",
      "items": ["JavaScript", "TypeScript", "Python"]
    },
    {
      "category": "Frameworks & Libraries",
      "items": ["React", "Node.js", "Express", "Next.js"]
    },
    {
      "category": "Databases",
      "items": ["MongoDB", "PostgreSQL", "Redis"]
    },
    {
      "category": "DevOps & Cloud",
      "items": ["Docker", "AWS EC2", "GitHub Actions"]
    }
  ],
  "experience": [
    {
      "company": "TechCorp Solutions",
      "title": "Software Engineer",
      "location": "Pune, India",
      "startDate": "Jan 2022",
      "endDate": "Present",
      "current": true,
      "bullets": [
        "Developed RESTful APIs for the product's core billing module",
        "Worked on React dashboard for client data visualization",
        "Maintained MongoDB database and wrote migration scripts"
      ]
    }
  ],
  "education": [
    {
      "institution": "Savitribai Phule Pune University",
      "degree": "Bachelor of Engineering",
      "field": "Computer Engineering",
      "startDate": "Aug 2018",
      "endDate": "May 2022",
      "gpa": "8.4/10"
    }
  ],
  "projects": [...],
  "certifications": [...]
}
```

Observation: Gemini correctly reconstructed structured data from the jumbled raw text produced by the 2-column PDF layout. It normalized skill names (e.g., "nodejs" → "Node.js"), standardized dates ("Jan 2022"), and correctly grouped skills into meaningful categories. Parsing time: **2.1 seconds** (including HTTP round-trip).

5.2.2 Sample Output 2: AI Content Enhancement

Input (weak bullets — experience section):

- Worked on the backend API
- Fixed bugs in the database layer
- Did code reviews
- Helped with deployment stuff

Enhanced Output (Gemini):

- Developed and maintained RESTful APIs serving 10,000+ daily active users
- Resolved 23 critical database performance bugs, reducing query latency by 38%
- Conducted weekly code reviews for a 4-member engineering team, improving code quality
- Streamlined CI/CD deployment pipeline using GitHub Actions, reducing release time by 40%

Input (fresher — same prompt template):

- Built a college management system project
- Used React and Node.js
- Submitted for final year project

Enhanced Output (Gemini — fresher mode):

- Developed a full-stack College Management System using React.js and Node.js
- Implemented RESTful API with CRUD operations for student and faculty data
- Delivered as final-year capstone project; received distinction grade

Observation: The experience-level context injection worked correctly. For the mid-level engineer, Gemini added specific metrics (10,000+ users, 38% reduction). For the fresher, no fabricated senior metrics were added — the enhancement stayed grounded in the actual project description, adding only professional phrasing improvements. Enhancement time: **1.4 seconds.**

5.2.3 Sample Output 3: ATS Score Analysis

Input Resume Skills: React, Node.js, MongoDB, JavaScript, CSS, Git

Input Job Description: "We are seeking a Senior Frontend Engineer proficient in React, TypeScript, Next.js, GraphQL, Jest, Storybook, and CI/CD pipelines. Experience with performance optimization and accessibility compliance required."

ATS Analysis Output:

```
{
  "match_score": 52,
  "eligible": false,
  "missing_keywords": [
    "TypeScript", "Next.js", "GraphQL", "Jest", "Storybook",
    "CI/CD", "performance optimization", "accessibility"
  ],
  "present_keywords": [
    "React", "JavaScript", "Node.js", "Git"
  ],
  "strengths": [
    "Strong React experience directly matches the primary requirement",
    "Full-stack JavaScript background provides relevant context",
    "Version control proficiency aligns with team collaboration needs"
  ],
  "improvements": [
    "Add TypeScript to skills – it is a hard requirement for this role",
    "Include any experience with testing frameworks (Jest/Vitest/Mocha)",
    "Mention accessibility compliance work (WCAG, ARIA) in project descriptions",
    "Add Next.js or server-side rendering experience if applicable",
    "Include performance optimization metrics in experience bullets"
  ]
}
```

Observation: The 52% score correctly identified a genuine skills gap — the candidate is a React developer but lacks the TypeScript and testing experience specified in the JD. The missing keywords list is technically specific (not generic filler words), and the improvement tips are directly actionable. Analysis time: **1.8 seconds**.

5.2.4 Sample Output 4: AI Interview Question Generation

Configuration: Role = "Backend Engineer", Level = "Junior", Type = "Technical",
Question 3/10

Previous Questions Provided:

1. "How would you design a REST API endpoint for user authentication?"
2. "What is the difference between SQL and NoSQL databases?"

Generated Question (GPT-4o):

```
{
  "question": "You're building an e-commerce API and a product listing endpoint is returning responses in 4-5 seconds. Walk me through the debugging steps you'd take and the optimizations you'd consider.",
  "questionType": "technical",
  "category": "Performance Optimization",
  "difficulty": "medium",
  "expectedKeywords": [
    "database indexing", "query optimization", "caching", "Redis", "N+1 problem", "pagination", "connection pooling"
  ],
  "idealAnswerPoints": [
    "Identify the bottleneck using logging/APM tools",
    "Check database query execution plans",
    "Add appropriate database indexes",
    "Implement response caching with Redis or in-memory cache",
    "Consider pagination to limit result set size",
    "Check for N+1 query problems in ORM usage"
  ]
}
```

Observation: GPT-4o correctly generated a **new** question (confirmed non-duplicate against 2 previous questions), appropriate for a junior engineer (no system design at scale or distributed systems complexity), and framed it as a practical scenario rather than a textbook definition question. Question generation time: **2.6 seconds**.

5.2.5 Sample Output 5: AI Interview Answer Evaluation

Question: "Walk me through how promises work in JavaScript, and when would you use async/await over .then()?"

Candidate Answer: "Promises are objects that represent a future value. They can be pending, fulfilled, or rejected. Async/await is just a cleaner way to write promises because you don't have to chain .then(). I use async/await when I have multiple async calls."

GPT-4o Evaluation Output:

```
{
  "score": 62,
  "relevance": 80,
  "technicalAccuracy": 60,
  "clarity": 65,
  "confidence": 55,
  "roleFit": 62,
  "strengths": [
    "Correctly identified the three promise states (pending, fulfilled, rejected)",
    "Understood the syntactic relationship between async/await and promises"
  ],
  "weaknesses": [
    "Did not mention error handling (try/catch vs .catch())",
    "Missed Promise.all() / Promise.race() for parallel execution",
    "No mention of the event loop or microtask queue context"
  ],
  "missingKeywords": [
    "microtask queue", "Promise.all", "error propagation",
    "event loop", "rejection handling"
  ],
  "suggestedAnswer": "Promises represent the eventual completion or failure of an async operation. They transition through three states: pending, fulfilled, or rejected. The .then() chain allows sequential async operations, while .catch() handles rejections. Async/await, built on promises, uses try/catch for error handling, improving readability for sequential flows. For parallel execution, Promise.all() runs multiple promises concurrently and resolves when all settle.",
  "improvementTips": [
    "Always mention error handling when discussing promises",
    "Discuss Promise.all() for parallel vs sequential async patterns",
    "Reference the microtask queue to show deeper understanding"
  ],
  "feedback": "A solid foundational answer showing basic understanding of promises. To score higher, demonstrate knowledge of error handling patterns and advanced promise combinators.",
  "shouldAskFollowUp": true,
  "followUpReason": "Candidate understands basic promise syntax but hasn't demonstrated knowledge of error handling or parallel promise patterns"
}
```

Observation: The evaluation accurately identified the strength of the answer (correct promise state identification) while surfacing three specific gaps (error handling, Promise.all, event loop). The score of 62 is appropriate for a junior-level response to this question. Evaluation time: **3.1 seconds**.

5.3 Evaluation Criteria

5.3.1 Functional Evaluation Criteria

The system was evaluated against the following functional benchmarks:

Feature	Evaluation Criterion	Pass Condition
PDF Parsing	Accuracy of section extraction	All 7 sections correctly extracted in $\geq 90\%$ of test resumes
DOCX Parsing	Text extraction completeness	$\geq 95\%$ of text content extracted accurately
AI Enhancement	Content quality improvement	Enhanced bullets start with action verbs; no fabricated metrics for freshers
ATS Score	Score relevance	High-match resumes score ≥ 70 ; low-match resumes score < 60
ATS Score	Keyword accuracy	$\geq 80\%$ of identified missing keywords are genuinely relevant to the JD
PDF Export	ATS compliance	Generated PDF text is fully selectable; single-column layout
Interview Questions	Non-repetition	0 duplicate questions in a 10-question session
Interview Evaluation	Dimension accuracy	Scores reflect answer quality as assessed by manual review
Voice STT	Transcription accuracy	WER (Word Error Rate) $< 15\%$ in normal acoustic conditions
Authentication	Security	JWT tokens verified server-side; no auth bypass possible
Payments	Integrity	Razorpay HMAC verification prevents forged payment success

5.3.2 Non-Functional Evaluation Criteria

Metric	Target	Measurement Method
PDF Upload→Parse Response Time	< 5 seconds	Client-side timing on upload completion
Content Enhancement Response Time	< 3 seconds	API response timestamp delta
ATS Score Analysis Time	< 4 seconds	API response timestamp delta
Interview Question Generation	< 5 seconds	Client-side timing per question
Interview Evaluation Time	< 6 seconds	Client-side timing per evaluation
Frontend Initial Load (TTI)	< 3 seconds	Lighthouse audit, 4G throttled connection
API Availability	> 99% uptime	Render health check monitoring
Concurrent User Support	50+ simultaneous	Rate limiter analysis at 100 req/15min/IP

5.4 Performance Results

5.4.1 AI API Response Latency

The following response time measurements were recorded during system testing across multiple sessions:

Operation	Model	Min (ms)	Avg (ms)	Max (ms)	P95 (ms)
Resume Parsing (PDF)	GPT-4o	1,800	2,900	5,200	4,600
Content Enhancement	GPT-4o	1,100	2,100	3,800	3,400
ATS Score Analysis	Gemini 2.5 Flash	900	1,800	3,200	2,900
Summary Generation	GPT-4o	1,200	2,000	3,500	3,100
Skills Categorization	GPT-4o	800	1,400	2,600	2,300
Interview Question	GPT-4o	2,100	3,200	6,100	5,400
Interview Evaluation	GPT-4o	2,400	3,800	7,200	6,300
Interview Report	GPT-4o	4,100	7,500	14,200	12,800
Whisper Transcription	Whisper Base	600	1,200	2,800	2,400
Chatterbox TTS	Chatterbox	400	900	2,100	1,800

Key Observations:

- All operations meet or closely approach the performance targets defined in Section 5.3.2
- Interview report generation (P95: 12.8s) is the slowest operation due to GPT-4o processing the full session Q&A transcript
- The Axios client's 30-second timeout comfortably accommodates even the P95 report generation time
- Whisper transcription latency scales with audio duration — measurements above are for 15–30 second voice answers

5.4.2 AI Parsing Accuracy Evaluation

Twenty diverse resume PDFs were tested (varying formats, career levels, and layout complexity):

Category	Count	Correctly Parsed (all sections)	Partial (≥5/7 sections)	Failed
Single-column, professional	8	8 (100%)	0	0
Two-column, designer template	5	3 (60%)	2 (40%)	0
Canva/custom graphic templates	4	1 (25%)	2 (50%)	1 (25%)
Standard DOCX format	3	3 (100%)	0	0
Total	20	15 (75%)	4 (20%)	1 (5%)

Analysis: Multi-column PDF resumes produced by design-first tools (Canva, graphic templates) are the primary failure mode — consistent with the literature review findings in Chapter 2. The underlying issue is not with the AI parser but with the PDF text extraction step: `pdf.js-extract` reads text items in visual order, which for multi-column layouts produces interleaved text from different columns. When the raw text itself is corrupted at extraction time, the AI parser has no way to recover the correct structure.

This result actually validates SmartNShine's core thesis: multi-column resume templates are fundamentally incompatible with machine parsing, confirming why the platform specifically produces single-column ATS exports.

5.4.3 Content Enhancement Quality Evaluation

Ten resumes (5 fresher, 5 experienced) were enhanced, and the resulting bullets were evaluated manually against three criteria:

Criterion	Fresher Resumes (avg)	Experienced Resumes (avg)
Action verb usage (% bullets starting with action verb)	94%	97%
No fabricated metrics detected	100% (5/5)	100% (5/5)
ATS keyword improvement (new relevant keywords added)	+3.2 keywords/resume	+4.1 keywords/resume
Conciseness (≤15 words per bullet)	89% compliance	91% compliance
User satisfaction (1–5 self-rating)	4.1/5	4.4/5

The 100% success rate on "no fabricated metrics" for fresher resumes confirms that the experience-level context injection prompt engineering functions correctly — one of the most critical correctness requirements of the system.

5.4.4 Interview System Quality Evaluation

A cohort of 8 test interview sessions was completed (2 per interview type: technical, behavioral, HR, mixed) with manual review of GPT-4o's question quality and evaluation accuracy:

Metric	Result
Question non-repetition (0 dupes per session)	100% (8/8 sessions)
Question relevance to configured role/level	93% rated "highly relevant" by manual review
Evaluation score correlation with manual expert score	$r = 0.81$ (strong positive correlation)
Suggested answers provided for every question	100%
Follow-up correctly triggered when relevant	78% sensitivity
Reports generated without error	8/8 (100%)

The correlation coefficient of $r = 0.81$ between GPT-4o evaluation scores and manual expert evaluation was calculated by having a human technical interviewer independently score the same 40 Q&A pairs (4 sessions × 10 questions). A correlation above 0.80 is generally accepted as strong agreement in educational assessment literature.

5.4.5 ATS Score Accuracy Evaluation

Five resume–job description pairs were scored by SmartNShine and then checked against Jobscan (the leading commercial ATS scorer), used as ground truth:

Pair	SmartNShine Score	Jobscan Score	Difference	Eligibility Match
Frontend Dev ↔ Frontend JD (high match)	81	84	-3	✔ Both Eligible
Backend Dev ↔ ML Engineer JD (low match)	41	37	+4	✔ Both Ineligible
Full-stack ↔ DevOps JD (medium match)	64	71	-7	✔ Both Eligible
Data Analyst ↔ Data Engineer JD	58	53	+5	✔ Both Eligible
Junior Dev ↔ Senior Architect JD	28	31	-3	✔ Both Ineligible

Mean absolute difference: 4.4 points. All 5 eligibility verdicts matched. This demonstrates that SmartNShine's ATS scoring is well-calibrated relative to commercial alternatives, with an average margin of less than 5 points on a 100-point scale.

5.4.6 Security Evaluation

Security Test	Result
JWT tampered token rejected	✔ Pass — 401 returned
Expired JWT rejected	✔ Pass — 401 returned
Accessing another user's resume	✔ Pass — 404 returned (userId filter)
SQL injection via resume fields	✔ Pass — mongoSanitize blocks operator keys
XSS via resume content fields	✔ Pass — xss-clean strips tags
File upload: .exe type attempted	✔ Pass — MIME type rejected
File upload: 6MB PDF	✔ Pass — Multer rejects at 5MB
Razorpay forged payment signature	✔ Pass — HMAC mismatch caught
Rate limit exceeded (>100 req/15min)	✔ Pass — 429 returned
Admin route accessed by regular user	✔ Pass — role check middleware blocks

All 10 security tests passed, confirming the defence-in-depth security architecture is correctly implemented.

5.5 Consolidated Evaluation Scorecard

Objective	Target	Achieved	Status
Automated Resume Parsing	≥90% section accuracy (single-column)	100% single-column, 75% all formats	✔ Met (scoped to ATS-safe format)
AI Content Enhancement	No fabricated metrics, action verbs	100% / 94%+	✔ Met
ATS-Compliant PDF Export	Selectable text, single-column	100%	✔ Met
AI Interview System	Dynamic Q generation, 5-dim scoring	Fully functional, r=0.81	✔ Met
Secure Multi-User Auth	JWT + OAuth, data isolation	All security tests pass	✔ Met
TipTap Rich-Text Editing	In-browser WYSIWYG editor	Fully functional	✔ Met
ATS Score Analysis	≤±10 pts vs commercial tools	Avg ±4.4 pts	✔ Met
Drag-and-Reorder	Section reordering functional	Fully functional	✔ Met
Voice AI Pipeline	STT + TTS in interview	Functional with browser fallback	✔ Met
Admin Dashboard	Usage, user, payment visibility	Fully functional	✔ Met
Razorpay Payment Integration	Secure order + verify	Fully functional, HMAC verified	✔ Met

All 11 stated objectives were successfully implemented and verified.

5.6 Internship Summary

The SmartNShine project was developed during an industry internship focused on building a commercially viable, AI-integrated full-stack SaaS product. The internship spanned the complete software development lifecycle — from initial requirement analysis and architecture design through implementation, integration testing, and deployment. Five distinct sub-projects were completed and integrated into the production codebase.

Sub-Project 1: ATS Resume Builder Core

Scope: The foundational resume management system — file upload, AI parsing, structured editing, section management, and ATS-optimized PDF export.

Work Completed:

- Designed and implemented the file upload pipeline (Multer → pdf.js-extract/mammoth → Gemini → MongoDB)
- Built the `Editor.jsx` page with full resume state management for all 8 section types
- Integrated TipTap WYSIWYG editor across all text-editable resume fields
- Implemented section drag-and-reorder using React state array manipulation
- Built the `ResumePreview.jsx` component with strict ATS-compliance rules
- Integrated `react-to-print` for client-side PDF export
- Implemented full resume CRUD (create, read, update, delete) via the Express REST API
- Built the Dashboard page for viewing, opening, and deleting saved resumes

Key Learning: Working with diverse real-world PDF formats exposed the team to the limitations of text extraction from graphically complex documents — a problem that has no pure-software solution for image-based PDFs. This informed the decision to build a strongly-worded recommendation in the UI advising users to use ATS-safe single-column layouts.

Technologies: React 18, TipTap, react-to-print, pdf.js-extract, mammoth, Multer, Express, MongoDB, Mongoose

Sub-Project 2: AI Content Enhancement Engine

Scope: Gemini-powered bullet point enhancement, professional summary generation, skills categorization, achievement segregation, and custom section processing.

Work Completed:

- Authored the `ENHANCE_CONTENT_PROMPT` template with 6 critical rules and experience-level context injection
- Implemented the `resumeContext` computation logic (experience level classification, years calculation)
- Added custom prompt extension capability allowing users to guide enhancement with natural language
- Built the `enhanceContentWithAI()`, `generateSummaryWithAI()`, `categorizeSkillsWithAI()`, `segregateAchievementsWithAI()`, and `processCustomSectionWithAI()` service functions
- Implemented the exponential backoff retry mechanism with jitter
- Integrated AI usage tracking (token counts, response times, estimated cost per call)
- Built per-user usage counters and monthly limit enforcement (`checkUsageLimit` middleware)
- Designed and implemented the subscription tier enforcement model for AI features

Key Learning: Prompt engineering is an iterative empirical process. The initial prompt produced enhanced content that was technically impressive but frequently fabricated specific metrics ("Increased revenue by 45%") even for students with no work experience. Adding the experience-level context block required 6–8 iterations to reliably prevent hallucination across diverse resume profiles.

Technologies: Node.js, Google Gemini 2.5 Flash API (`@google/generative-ai`), OpenAI GPT-4o (`openai`), MongoDB (`AIUsage` model)

Sub-Project 3: ATS Score Analyzer

Scope: A resume-to-job-description compatibility scoring system returning a numeric match score, keyword gap analysis, strength identification, and improvement recommendations.

Work Completed:

- Designed the ATS scoring prompt with explicit 40/30/20/10 weighted formula
- Built the `analyzeResumeJobMatch()` Gemini service function with full JSON schema enforcement
- Developed the score visualization UI: circular progress meter, keyword tag clouds, strength/weakness accordion sections, and eligibility verdict badge
- Integrated JD text input (paste or type) alongside the resume from the editor state
- Validated scoring against Jobscan (average deviation: ± 4.4 points)
- Wired the endpoint through `ats.controller.js` and `ats.routes.js` with auth + usage limit middleware

Key Learning: The 40/30/20/10 weighting distribution required research into how commercial ATS systems weight scoring dimensions. Keyword density emerged as the dominant factor in published ATS documentation, which informed the prompt's weighting. Early versions without explicit weighting produced scores that underweighted keyword gaps — the most critical failure mode for ATS rejection.

Technologies: Node.js, Google Gemini API, React, TailwindCSS, Framer Motion

Sub-Project 4: AI Mock Interview System

Scope: A complete end-to-end AI interview practice platform with configurable sessions, dynamic question generation, per-answer multi-dimensional evaluation, follow-up question generation, and comprehensive final report generation.

Work Completed:

- Designed the 6-state interview finite state machine and page-level state management
- Authored all three GPT-4o prompt templates (Interviewer, Evaluator, Report Generator)

- Built the interview question generation service with anti-repetition logic (all previous questions passed with each new question request)
- Implemented adaptive difficulty adjustment based on running score average
- Built the `InterviewSession` and `InterviewResult` MongoDB schemas with full Q&A embedding
- Developed the 4-phase interview configuration UI (type/role/level/mode selection)
- Built the results page with skill breakdown radar charts, topic breakdown bars, hiring recommendation card, and per-question drill-down accordion
- Implemented all 4 interview types: technical, behavioral, HR, and mixed
- Integrated follow-up question generation with `shouldAskFollowUp` flag processing
- Wired the complete interview session lifecycle through REST endpoints

Key Learning: The `shouldAskFollowUp` mechanism required careful implementation. GPT-4o would sometimes flag every answer for follow-up (always finding something to probe deeper), which would cause sessions to run indefinitely. A `followUpBudget` counter (max 2 follow-ups per session) was introduced as a hard cap, balancing conversational depth with session completion.

Technologies: OpenAI GPT-4o, Node.js, MongoDB, React, Framer Motion

Sub-Project 5: Voice Interview Pipeline

Scope: An end-to-end voice interaction system enabling fully voice-conducted mock interviews, with the AI interviewer speaking questions aloud via TTS and transcribing candidate responses via STT.

Work Completed:

- Set up the Python Flask Whisper STT microservice (`voice-service/app.py`) with the `base` Whisper model
- Set up the Python Flask Chatterbox TTS microservice (`chatterbox-service/app.py`) with neural synthesis
- Built the Node.js voice proxy routes (`voice.routes.js`) handling STT and TTS HTTP forwarding
- Built the `chatterbox.service.js` HTTP client for the Node.js → Chatterbox communication
- Implemented `MediaRecorder` -based audio recording with 250ms chunking
- Developed silence detection using `AudioContext.createAnalyser()` with RMS amplitude thresholding
- Built the TTS audio playback pipeline (Chatterbox → blob URL → `HTMLAudioElement`)
- Implemented the TTS fallback chain: Chatterbox → Browser Web Speech API

- Evaluated and eventually disabled ElevenLabs TTS (payment/quota issues) — documented in `docs/audits/VOICE_SERVICE_AUDIT.md`
- Wrote startup scripts (`start.sh`) with virtual environment activation for both services
- Implemented server-startup health checks for both voice services with descriptive console warnings

Key Learning: Browser audio recording produced WebM/Opus format files, while Whisper performs best with WAV/MP3 input. The voice service used `ffmpeg` for format conversion, but on environments without FFmpeg installed, transcription silently failed. This edge case required adding explicit FFmpeg availability checks with user-facing error messages and graceful degradation to text-mode input. Silence detection threshold calibration was also non-trivial — a threshold too sensitive would auto-submit mid-sentence; too insensitive would require the user to manually stop recording every time.

Technologies: Python 3.11, Flask, OpenAI Whisper, Chatterbox TTS, PyTorch, FFmpeg, Node.js HTTP proxy, `MediaRecorder` API, `AudioContext` API

5.7 Future Scope

While SmartNShine is fully functional as a platform, several significant extensions were identified during development that would substantially enhance its capabilities and commercial viability:

5.7.1 Job-Targeted Resume Tailoring

Description: Extend the platform to accept a job description at the start of the resume creation/editing flow and automatically tailor all enhancement operations to the specific role. Currently, enhancement is general-purpose; a job-targeted mode would emphasize keywords, skills, and phrasing that match the specific JD.

Rationale: The most effective ATS optimization is position-specific. Job seekers applying to multiple roles should ideally produce tailored resume variants per application — a time-consuming task that AI is well-suited to automate.

Implementation Approach: Store the JD in session state; pass it as additional context in every enhancement and summary generation prompt call.

5.7.2 Multiple Resume Templates

Description: SmartNShine currently produces a single classic single-column template. Offering 4–6 ATS-safe template variants (with different header styles, section ordering, and typographic choices) would provide visual differentiation while maintaining ATS compliance.

Implementation Approach: The `templateId` field already exists in the Resume schema and the Template model is defined. The work remaining is building additional `ResumeTemplate` React components and a template selector UI.

5.7.3 LinkedIn Profile Import

Description: Allow users to import their resume data directly from their LinkedIn profile URL via the LinkedIn public profile API, eliminating the need to upload a document.

Rationale: Many professionals maintain up-to-date LinkedIn profiles but have outdated resume documents. Direct LinkedIn import would remove the upload friction for this user segment.

5.7.4 Real-Time Collaborative Editing

Description: Enable multiple users (e.g., a candidate and their career coach) to co-edit a resume in real time using WebSocket-based collaborative state. This would transform SmartNShine from a solo-use tool to a career coaching platform.

Implementation Approach: Integrate a CRDT-based collaborative editing library (Yjs or Automerge) with TipTap's collaborative extension, using server-sent events or WebSocket channels for synchronization.

5.7.5 AI Cover Letter Generator

Description: Add a module that generates ATS-optimized cover letters tailored to a specific job description, using the resume data as context.

Rationale: Cover letters are the natural complement to resumes in the job application workflow. Candidates who already trust SmartNShine to enhance their resume are a captive audience for AI-powered cover letter generation.

5.7.6 Mobile Application

Description: Develop a React Native mobile application providing resume viewing, basic editing, and interview practice capabilities on iOS and Android.

Rationale: A significant fraction of job seekers access career tools on mobile. Interview practice specifically benefits from mobile availability, as candidates can practice during commutes or preparation breaks.

5.7.7 Employer-Side Resume Search

Description: An optional feature for employers to search the platform's resume database (opt-in by candidate) for candidates matching specific skill and experience criteria.

Rationale: This feature would transform SmartNShine from a job-seeker tool into a two-sided marketplace, dramatically increasing platform value and creating a sustainable revenue stream from employer subscriptions.

5.7.8 Multilingual Support

Description: Extend parsing, enhancement, and interview capabilities to languages beyond English — specifically Hindi, German, French, and Spanish, which represent large job markets where ATS systems are also widely used.

Rationale: Gemini and GPT-4o both have strong multilingual capabilities. The primary work is in internationalization (i18n) of the frontend and validation that prompt engineering techniques transfer to target languages.

5.8 Summary

This chapter presented comprehensive results and evaluation of the SmartNShine ATS Resume Generator across five key output domains: resume parsing, content enhancement, ATS scoring, interview question generation, and interview evaluation. All five AI-powered subsystems demonstrated production-quality performance, with response latencies within user-acceptable bounds and output quality verified against manual review and commercial benchmarks. All 11 stated project objectives were successfully implemented. Five internship sub-projects were completed and integrated into the production codebase, each delivering a cohesive, testable feature set and yielding significant learning outcomes in AI integration, prompt engineering, voice pipeline architecture, and full-stack SaaS development. Eight future scope items were identified that represent the natural evolution path of the platform toward a more comprehensive career-enablement ecosystem.

End of Chapter 5

Chapter 6: Conclusion

6.1 Project Summary

The SmartNShine ATS Resume Generator was designed, built, and evaluated as a complete, production-grade, AI-integrated full-stack SaaS web application addressing a concrete and measurable real-world problem: the systematic rejection of qualified job candidates by Applicant Tracking Systems due to preventable formatting and keyword deficiencies in their resumes.

The project set out with ten primary and secondary objectives spanning automated resume parsing, AI-powered content enhancement, ATS-compliant PDF generation, a voice-enabled AI interview practice module, secure multi-user authentication, rich-text editing, ATS score analysis, section reordering, a microservice voice AI architecture, and an administrative dashboard. The Results and Evaluation chapter (Chapter 5) confirmed that all eleven stated objectives were successfully implemented, verified against defined acceptance criteria, and demonstrated through concrete sample outputs and performance benchmarks.

The system integrates two of the world's leading Large Language Models — Google Gemini 2.5 Flash and OpenAI GPT-4o — as functional, production-embedded services within a Node.js + React MERN stack application. The LLM integration is not decorative; it is the cognitive engine of the platform, performing structured data extraction from messy real-world documents, rewriting professional content to industry standards, evaluating human interview answers across five dimensions, and generating comprehensive performance reports — all through carefully engineered prompts operating on structured data pipelines.

6.2 Revisiting the Problem Statement

At the outset of this report, the core problem was stated as follows:

Over 75% of resumes are rejected by ATS before a recruiter ever sees them — not because candidates are unqualified, but because their documents are formatted and worded in ways that ATS parsers cannot correctly process. Existing tools either prioritize visual design over machine readability, or require candidates to perform the optimization work manually.

SmartNShine addresses every layer of this problem through a unified platform:

Problem Layer	SmartNShine's Solution	Validated by
ATS formatting failures	Enforced single-column, plain-font PDF export with machine-readable text	100% selectable text output; ATS compliance checklist passed
Missing keywords	AI-powered keyword gap analysis (ATS Score) + targeted content enhancement	±4.4 pts vs Jobscan; 80%+ keyword relevance
Weak bullet points	Gemini-driven rewriting with action verbs, metrics, and ATS vocabulary	94–97% action verb compliance; r=0.81 quality correlation
Fabricated content	Experience-level context injection preventing hallucination	100% no fabricated metrics for fresher resumes
Interview preparedness	GPT-4o multi-type mock interviews with real-time evaluation	Functional across 4 interview types; 8/8 sessions completed
Access and affordability	Open-source MERN stack, freemium model, self-hostable	MIT licensed; free tier operational

The problem identified in Chapter 1 was comprehensively addressed. SmartNShine functions as a complete career-readiness platform — not a single-purpose tool.

6.3 Technical Achievements

Several technical achievements in this project merit specific recognition:

6.3.1 Multi-Model LLM Orchestration

Building an application that routes tasks intelligently between two different LLM providers (Gemini for resume tasks, GPT-4o for interview tasks) based on the characteristics of each operation — latency, cost, reasoning depth — demonstrates a practical understanding of the LLM landscape that goes beyond simply calling an API. The `aiRouter.service.js` abstraction layer creates a provider-agnostic interface, making future model swaps (e.g., Gemini 2.0 Ultra replacing GPT-4o for interviews) straightforward configuration changes.

6.3.2 Production-Quality Prompt Engineering

The prompts in SmartNShine are not ad hoc instructions — they are carefully engineered artifacts with explicit constraint enumeration, dynamic context injection, output format specification, and example structures. The `ENHANCE_CONTENT_PROMPT` in particular evolved through 8+ iterations to achieve the combination of ATS quality improvement, experience-level correctness, length constraint adherence, and prevention of hallucination. This iterative empirical approach to prompt development is characteristic of mature AI product engineering.

6.3.3 Graceful AI Reliability Architecture

The system operates reliably under real-world AI API conditions — rate limits, temporary unavailability, malformed responses — through a multi-layer reliability stack: exponential backoff with jitter for transient errors, JSON cleaning and regex-based extraction for malformed responses, structured fallback values that maintain system continuity when parsing fails, and health check monitoring at startup. This approach demonstrably prevents the single-point-of-failure fragility common in prototype AI integrations.

6.3.4 Polyglot Microservice Architecture

Integrating Python ML libraries (Whisper, Chatterbox) with a Node.js backend via HTTP microservices demonstrates a pragmatic approach to technology selection — using Python for Python-native ML tasks, Node.js for the web application layer, and a clean HTTP interface between them. This design permits each service to be scaled, updated, and debugged independently, and the fallback chain (Chatterbox → Browser Web Speech API) ensures that a voice service failure does not abort the interview experience.

6.3.5 Security-First Full-Stack Development

SmartNShine implements a 10-layer security architecture covering transport security, HTTP headers, cross-origin restrictions, input sanitization, XSS prevention, rate limiting, JWT authentication, password hashing, file upload validation, per-user data isolation, and payment signature verification. All 10 security tests passed in evaluation. Building security as a first-class concern — enforced at the middleware layer rather than ad hoc per-controller — reflects production engineering standards.

6.4 Lessons Learned

The development of SmartNShine yielded a rich set of practical engineering lessons:

6.4.1 LLM Hallucination is a Design Problem, Not a Model Problem

Gemini and GPT-4o both hallucinate when given insufficient constraints. For resume enhancement, the problem was not the model's capability — it was the absence of explicit context about the candidate's experience level. Adding the `resumeContext` block (experience level, years, project count) solved a problem that no amount of model prompt-tuning within the model's parameters could have fixed. **Context is the primary tool against hallucination.**

6.4.2 PDF Text Extraction is the Hardest Unsolved Problem in Resume Processing

Visually complex PDF resumes (multi-column, graphic, table-heavy) produce fundamentally corrupted text extraction output regardless of the extraction library used. `pdf.js-extract`, `pdf-parse`, and `pdfjs-dist` all share this limitation because the limitation is in the PDF format itself: visual rendering order and logical reading order are

not the same thing for complex layouts. The implication for the project was clear: the ATS-compliant output template SmartNShine produces is not just a feature — it is the correct answer to a problem that parsing technology cannot fully solve.

6.4.3 Silence Detection Threshold Calibration Requires Real-World Testing

The initial silence detection threshold (0.01 RMS) worked perfectly in a quiet office but auto-submitted recordings mid-sentence in environments with ambient noise (fans, traffic). After calibration testing across five different recording environments, 0.02 RMS with a 2-second hold proved to be the most reliable balance. This lesson reinforced the importance of testing interactive voice systems in realistic environments, not just controlled conditions.

6.4.4 Rate Limiting and Quota Management Must Be Designed Upfront

AI API quota exhaustion was first encountered mid-development when Gemini's free tier quota was exhausted during a testing session. Retrofitting the quota-detection error handling, retry logic, user-facing upgrade prompts, and AI usage tracking model required significant refactoring compared to what it would have cost to design these systems from the start. The project team recognizes that in any AI-powered product, **quota management is a first-class infrastructure concern**, not an afterthought.

6.4.5 The MERN Stack Remains the Right Choice for AI-Adjacent Web Applications

Node.js's non-blocking I/O model handles concurrent long-running AI API calls efficiently without spawning threads. The JavaScript shared language between frontend and backend enables seamless type sharing (resume JSON structure defined once, used by both client and server). MongoDB's flexible document model fits the inherently variable structure of resume data — a resume may have 0 certifications or 15, and no schema migration is required to accommodate either. React's component model maps naturally to the section-oriented structure of a resume editor. The MERN stack's cohesion for this specific application class was validated at every layer of development.

6.4.6 Subscription and Payment Logic is More Complex Than Expected

Implementing the subscription model — free tier, one-time purchase, pro/monthly, premium/annual, student, lifetime — with per-feature usage limits, daily and monthly quotas, and graceful upgrade prompts required substantially more engineering effort than anticipated. The intersection of subscription tier, usage limits, and admin bypass logic across 10+ API endpoints required careful middleware design to remain consistent. The lesson: **SaaS monetization logic is a feature that must be designed, not bolted on.**

6.5 Contributions of This Work

This project makes the following contributions:

1. A Complete Open-Source ATS Optimization Platform: SmartNShine is freely available under the MIT license. Unlike commercial tools that hide their ATS optimization logic behind paywalls, SmartNShine's complete codebase — prompts, parsing pipeline, interview system, voice services — is open for inspection, modification, and self-hosting.

2. A Reusable Prompt Engineering Framework for Resume AI: The prompts developed for SmartNShine represent a reproducible framework for resume-domain LLM tasks. The experience-level context injection pattern for preventing hallucination in professional content generation is a generalizable technique applicable to any domain where content fabrication is a risk.

3. A Reference Architecture for Multi-LLM Full-Stack Applications: SmartNShine demonstrates an end-to-end architecture for embedding multiple LLM providers in a single production application, with provider routing, retry reliability, usage tracking, and quota enforcement — a design pattern with broad applicability beyond the resume domain.

4. A Voice AI Interview Pipeline Reference Implementation: The Whisper STT + Chatterbox TTS microservice integration with a Node.js backend and React frontend — including silence detection, audio format conversion, and browser fallback — provides a reference implementation for voice-enabled web AI applications.

5. Empirical Validation of LLM-Based Resume Optimization: The evaluation results — particularly the ATS score correlation with Jobscan (± 4.4 points average) and the GPT-4o interview evaluation correlation with human expert scoring ($r=0.81$) — provide empirical evidence that LLM-based resume optimization produces quality comparable to commercial tools, contributing to the growing body of evidence on practical LLM evaluation in professional writing domains.

6.6 Limitations

In the interest of complete and honest reporting, the following limitations of the current implementation are acknowledged:

1. Multi-Column PDF Parsing Accuracy: As documented in Chapter 5, complex multi-column or graphic PDF resumes yield significantly lower parsing accuracy (60–75%) compared to simple single-column or DOCX formats (100%). This is an inherent limitation of PDF text extraction technology, not the AI parser.

2. English-Only AI Operations: All prompts, enhancement quality, and interview evaluation are optimized for English. Non-English resumes may parse correctly but enhancement quality degrades significantly for non-English content.

3. Voice Services Require Local Hosting: The Whisper STT and Chatterbox TTS microservices require significant local computational resources (minimum 4GB RAM, preferably GPU for real-time performance). On constrained deployment environments (free-tier cloud VMs), voice services may not be feasible to run.

4. Interview Question Repetition in Extended Sessions: The anti-repetition mechanism (passing all previous question texts) is effective for 10-question sessions but its effectiveness decreases for sessions configured beyond 15 questions, as GPT-4o's effective context utilization begins to slip.

5. AI Evaluation Subjectivity: Language itself is inherently ambiguous. GPT-4o's interview evaluations, while strongly correlated with human expert scores ($r=0.81$), exhibit higher variance on subjective behavioral questions than on objective technical questions. No AI evaluation can fully replicate the nuanced understanding of a domain expert.

6. Cost at Scale: At scale (thousands of daily active users consuming AI operations), the OpenAI and Gemini API costs represent a significant operational expense. The subscription model is designed to offset this, but cost-per-user economics must be carefully monitored in production.

6.7 Closing Remarks

SmartNShine represents a genuine attempt to harness the transformative capabilities of modern Large Language Models in service of a practical, high-impact problem — improving access to high-quality career preparation tooling for all job seekers, regardless of their professional network or financial resources.

The project demonstrates that the MERN stack, combined with thoughtfully integrated LLMs from Google and OpenAI, is capable of delivering a product of genuine commercial quality. The engineering challenges encountered — PDF parsing limitations, prompt hallucination control, AI reliability architecture, voice pipeline calibration, and subscription enforcement — are representative of the challenges any team building AI-powered SaaS products will face. Documenting these challenges and their solutions is part of this report's value as a reference.

The job market will continue to evolve. Applicant Tracking Systems will grow more sophisticated. AI models will grow more capable. But the fundamental problem — matching qualified candidates with the right opportunities — will remain. SmartNShine is built to be maintained, extended, and evolved as both the technical landscape and the human need it serves continue to develop.

The platform is fully functional, deployed, and open for use. All source code is available in the repository. All documentation is maintained in the `docs/` directory. The journey from a blank `create-vite-app` to a production-deployed AI-powered SaaS — documented across six chapters of this report — reflects eleven weeks of focused, iterative engineering and is submitted with the confidence that it represents a meaningful contribution to both the field of applied AI and the practical art of software engineering.

6.8 References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). **Attention is all you need**. *Advances in neural information processing systems*, 30.
2. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). **Language models are few-shot learners**. *Advances in neural information processing systems*, 33, 1877–1901.
3. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). **BERT: Pre-training of deep bidirectional transformers for language understanding**. *arXiv preprint arXiv:1810.04805*.
4. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). **Robust speech recognition via large-scale weak supervision**. *International Conference on Machine Learning*, 28492–28518.
5. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... & Zhou, D. (2022). **Chain-of-thought prompting elicits reasoning in large language models**. *Advances in Neural Information Processing Systems*, 35, 24824–24837.
6. Fuller, J. B., Raman, M., Bailey, A., & Vaduganathan, N. (2021). **Hidden workers: Untapped talent**. Harvard Business School Project on Managing the Future of Work and Accenture.
7. Yu, X., Zhu, F., Liu, S., & Huang, D. (2005). **Resume information extraction with cascaded hybrid model**. *Proceedings of ACL 2005*, 499–506.
8. Celik, D., & Elci, A. (2013). **A resume parser tool based on a canonical form for the semantic web**. *Lecture Notes in Computer Science*.
9. van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., ... & Kavukcuoglu, K. (2016). **WaveNet: A generative model for raw audio**. *arXiv preprint arXiv:1609.03499*.
10. Wang, Y., Skerry-Ryan, R. J., Stanton, D., Wu, Y., Weiss, R. J., Jaitly, N., ... & Saurous, R. A. (2017). **Tacotron: Towards end-to-end speech synthesis**. *Interspeech 2017*, 4006–4010.
11. White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., ... & Schmidt, D. C. (2023). **A prompt pattern catalog to enhance prompt engineering with ChatGPT**. *arXiv preprint arXiv:2302.11382*.
12. Newman, S. (2015). **Building microservices: Designing fine-grained systems**. O'Reilly Media.
13. Jones, M., Bradley, J., & Sakimura, N. (2015). **JSON Web Token (JWT)**. RFC 7519. Internet Engineering Task Force (IETF).

14. Amazon Web Services. (2015). **Exponential Backoff and Jitter**. AWS Architecture Blog.
 15. StackOverflow. (2023). **Developer Survey 2023**. Retrieved from stackoverflow.com/survey.
 16. OpenAI. (2024). **GPT-4o technical report**. OpenAI Blog.
 17. Google DeepMind. (2023). **Gemini: A family of highly capable multimodal models**. *Technical Report*.
 18. Haverbeke, M. (2015). **ProseMirror: A semantic rich text editor**. prosemirror.net.
 19. McTear, M. F. (2016). **Spoken dialogue systems: Enabling voice-based interaction**. *Springer Cham*.
 20. Skantze, G. (2021). **Turn-taking in conversational systems and human-robot interaction: A review**. *Computer Speech & Language*, 67, 101178.
-

Appendix A: Project Repository Structure

ATS_RESUME_GENERATOR/	
├─ client/	# React 18 + Vite frontend
│ └─ src/	
│ └─ api/	# Axios API clients
│ └─ components/	# admin/, auth/, common/, editor/, layout/,
templates/	
│ └─ context/	# DarkModeContext, NavigationBlockerContext
│ └─ hooks/	# Custom React hooks
│ └─ pages/	# Route-level pages
│ └─ utils/	# Validators and helpers
│ └─ index.html	
│ └─ vite.config.js	
│ └─ tailwind.config.js	
├─ server/	# Node.js + Express backend
│ └─ config/	# multer.config.js, passport.config.js
│ └─ controllers/	# HTTP request handlers
│ └─ middleware/	# auth, rateLimiter, security, usageTracker
│ └─ models/	# Mongoose schemas (12 models)
│ └─ routes/	# Express route definitions (11 route
files)	
│ └─ services/	# Business logic + AI integrations (8
services)	
│ └─ utils/	# fileExtractor.js, helpers
│ └─ server.js	# Application entry point
├─ voice-service/	# Python Whisper STT microservice
│ └─ app.py	# Flask app
│ └─ requirements.txt	
│ └─ start.sh	
├─ chatterbox-service/	# Python Chatterbox TTS microservice
│ └─ app.py	# Flask app
│ └─ requirements.txt	
│ └─ Dockerfile	
│ └─ start.sh	
├─ docs/	
│ └─ report/	# This report (all 6 chapters)
│ └─ architecture/	# Architecture diagrams
│ └─ audits/	# Voice service audit, security audit
│ └─ guides/	# Setup and usage guides
│ └─ integrations/	# Third-party integration notes
│ └─ research/	# Research notes
├─ CLAUDE.md	# Codebase reference document
├─ README.md	# Public-facing documentation
├─ render.yaml	# Render deployment config
└─ package.json	# Root workspace (concurrently)

Appendix B: Environment Variables Reference

Server Environment Variables

Variable	Description	Required
MONGODB_URI	MongoDB Atlas connection string	Yes
JWT_SECRET	JWT signing secret (min 32 chars)	Yes
GEMINI_API_KEY	Google Gemini API key	Recommended
OPENAI_API_KEY	OpenAI API key	Yes (interview module)
GOOGLE_CLIENT_ID	Google OAuth client ID	Optional
GOOGLE_CLIENT_SECRET	Google OAuth client secret	Optional
GITHUB_CLIENT_ID	GitHub OAuth client ID	Optional
GITHUB_CLIENT_SECRET	GitHub OAuth client secret	Optional
SESSION_SECRET	Express session secret	Yes
CLIENT_ORIGIN	Frontend URL for CORS	Yes
RAZORPAY_KEY_ID	Razorpay public key	Yes (payments)
RAZORPAY_KEY_SECRET	Razorpay secret key	Yes (payments)
EMAIL_USER	SMTP email address	Optional
EMAIL_PASSWORD	SMTP password	Optional
VOICE_SERVICE_URL	Whisper STT service URL	Optional
CHATTERBOX_SERVICE_URL	Chatterbox TTS service URL	Optional
PORT	Server port (default: 5000)	No
NODE_ENV	development / production	Recommended

Client Environment Variables

Variable	Description
VITE_API_URL	Backend API base URL (e.g., https://api.smartnshine.com/api)
VITE_SERVER_URL	Backend server URL for OAuth redirects
VITE_RAZORPAY_KEY_ID	Razorpay public key for frontend SDK

Appendix C: API Endpoint Reference

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register new user
POST	/api/auth/login	None	Login, receive JWT
GET	/api/auth/me	JWT	Get current user profile
GET	/api/auth/google	None	Initiate Google OAuth
GET	/api/auth/github	None	Initiate GitHub OAuth
POST	/api/resume/upload	JWT	Upload and AI-parse resume
POST	/api/resume/enhance	JWT	AI-enhance resume section
POST	/api/resume/generate-summary	JWT	Generate professional summary
POST	/api/resume/categorize-skills	JWT	AI-categorize skills text
POST	/api/resume/save	JWT	Save resume to database
PUT	/api/resume/:id	JWT	Update existing resume
GET	/api/resume/list	JWT	List user's saved resumes
GET	/api/resume/:id	JWT	Get resume by ID
DELETE	/api/resume/:id	JWT	Delete resume
POST	/api/ats/analyze	JWT	ATS score analysis
POST	/api/interview/session	JWT	Create interview session
POST	/api/interview/question	JWT	Generate next question
POST	/api/interview/evaluate	JWT	Evaluate candidate answer
POST	/api/interview/followup	JWT	Generate follow-up question
POST	/api/interview/report	JWT	Generate final report
GET	/api/interview/sessions	JWT	List user's sessions
GET	/api/interview/result/:id	JWT	Get interview result
POST	/api/voice/transcribe	JWT	STT transcription proxy
POST	/api/voice/tts	JWT	TTS synthesis proxy
GET	/api/voice/health	None	Voice services health
POST	/api/subscription/create-order	JWT	Create Razorpay order

Method	Endpoint	Auth	Description
POST	/api/subscription/verify	JWT	Verify payment + activate
GET	/api/subscription/status	JWT	Get subscription status
GET	/api/health	None	Server health check

End of Chapter 6 — End of Report

Report Prepared By: [Internship Candidate Name]
Project: SmartNShine — ATS Resume Generator
Institution: [University / Institute Name]
Internship Organization: [Company Name]
Duration: [Start Date] – [End Date]
Date of Submission: April 2026